Επίλυση Προβλημάτων

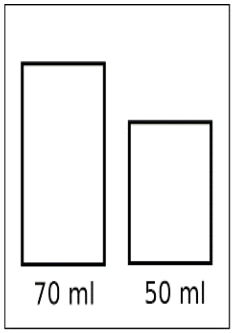
Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

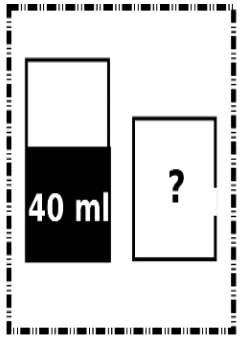
Αλγόριθμοι τυφλής Αναζήτησης

- Οι *αλγόριθμοι τυφλής αναζήτησης* (*blind search algorithms*) εφαρμόζονται σε προβλήματα στα οποία δεν υπάρχει πληροφορία που να επιτρέπει την αξιολόγηση των καταστάσεων του χώρου αναζήτησης.
- **Παράδειγμα:** Έστω το πρόβλημα των ποτηριών:

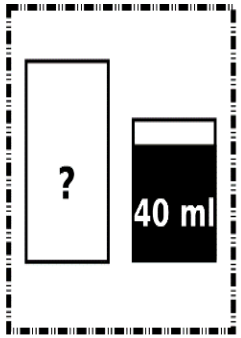
Αρχική και τελικές καταστάσεις του προβλήματος των ποτηριών.



Αρχική Κατάσταση



Τελική Κατάσταση



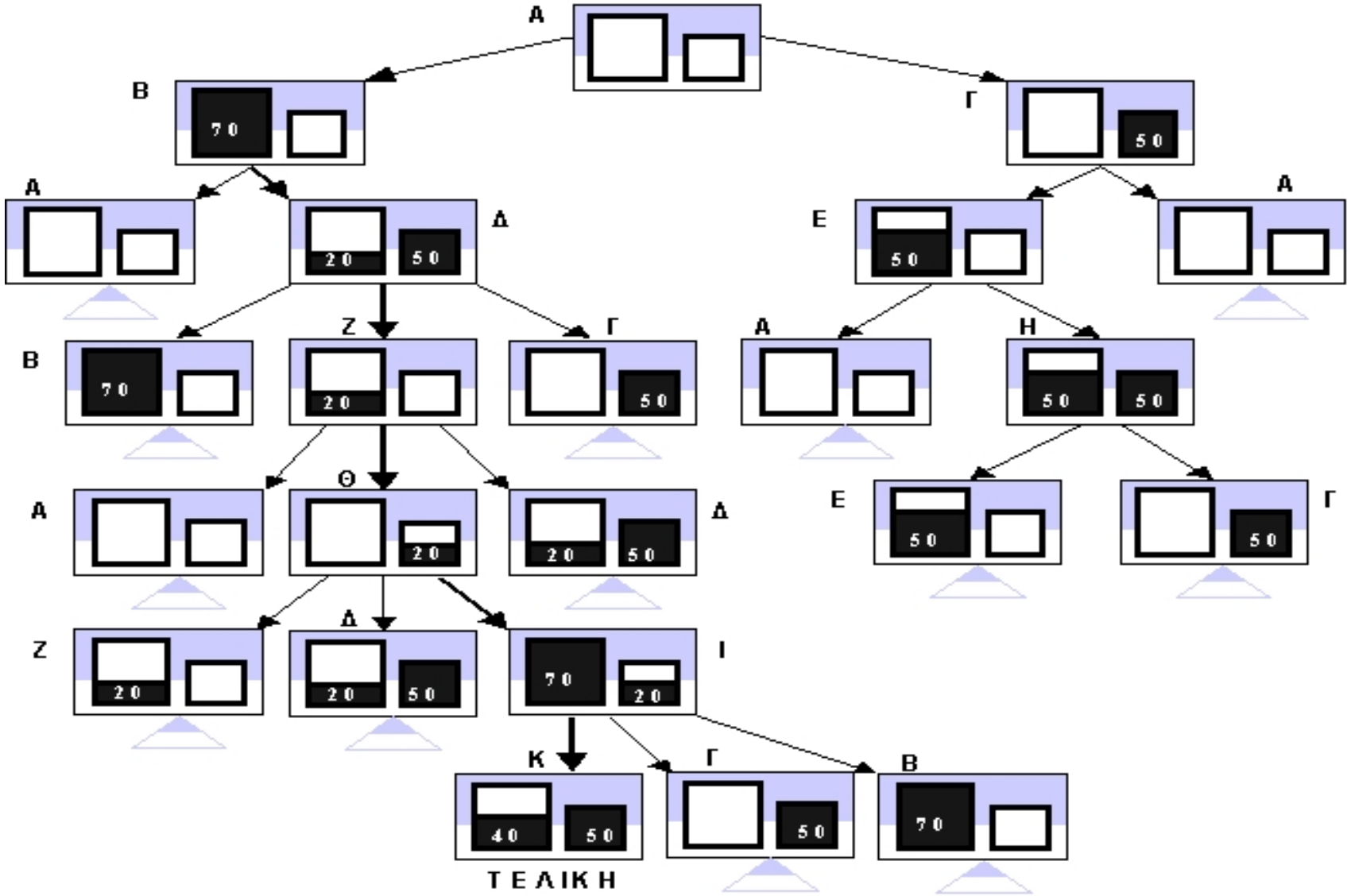
Τελική Κατάσταση

Παράδειγμα
Το πρόβλημα των ποτηριών:

- **Τελεστής:** Γέμισε το ποτήρι των X ml μέχρι το χείλος από τη βρύση
  - **Προϋποθέσεις:** Το ποτήρι των X ml έχει 0 ml
  - **Αποτελέσματα:** Το ποτήρι των X ml έχει X ml
- **Τελεστής:** Γέμισε το ποτήρι των X ml από το ποτήρι των Y ml
  - **Προϋποθέσεις**
    - Το ποτήρι των X ml έχει Z ml
    - Το ποτήρι των Y ml έχει W ml (W≠0)
- **Αποτελέσματα**
  - Το ποτήρι των X ml έχει X ml και το ποτήρι των Y ml έχει W-(X-Z), αν W≥X-Z

  ή
  - Το ποτήρι των X ml έχει Z+W ml και Το ποτήρι των Y ml έχει 0, αν W<X-Z
- **Τελεστής:** Άδειασε το ποτήρι των X ml στο νεροχύτη
  - **Προϋποθέσεις:** Το ποτήρι έχει περιεχόμενο
  - **Αποτελέσματα**Το ποτήρι των X ml έχει 0 ml

Ποτήρια: δένδρο αναζήτησης



Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

Αναζήτηση πρώτα σε βάθος (Depth-First Search DFS)

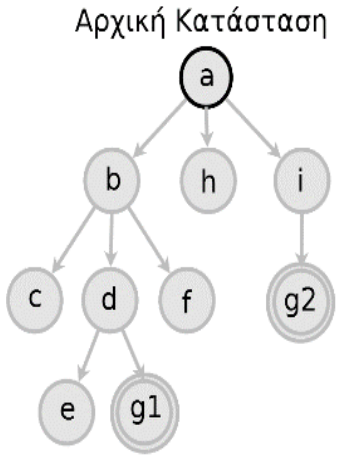
Ο αλγόριθμος *πρώτα σε βάθος* επιλέγει προς επέκταση την κατάσταση που βρίσκεται πιο βαθιά στο δένδρο.

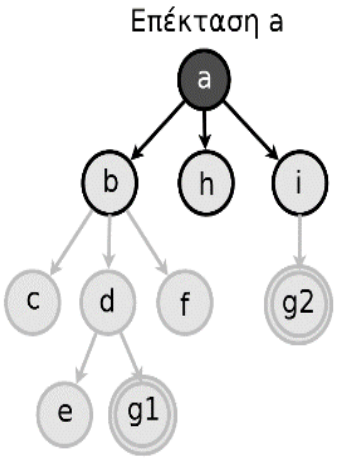
1. Βάλε την αρχική κατάσταση στο μέτωπο της αναζήτησης.
2. Αν το μέτωπο της αναζήτησης είναι κενό τότε σταμάτησε.
3. Βγάλε την πρώτη κατάσταση από το μέτωπο της αναζήτησης.
4. Αν είναι η κατάσταση μέλος του κλειστού συνόλου τότε πήγαινε στο βήμα 2.
5. Αν η κατάσταση είναι μία από τις τελικές, τότε ανέφερε τη λύση.
6. Αν θέλεις και άλλες λύσεις πήγαινε στο βήμα 2. Αλλιώς σταμάτησε.
7. Εφάρμοσε τους τελεστές μετάβασης για να βρεις τις καταστάσεις-παιδιά.
8. Βάλε τις καταστάσεις-παιδιά στην αρχή του μετώπου της αναζήτησης.
9. Βάλε την κατάσταση-γονέα στο κλειστό σύνολο.
10. Πήγαινε στο βήμα 2.

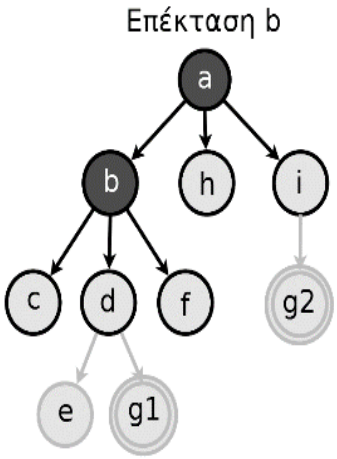
Αναζήτηση πρώτα σε βάθος (Depth-First Search DFS)

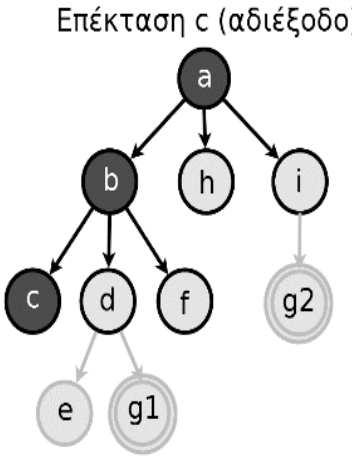
```
algorithm dfs(InitialState, FinalStates)
    begin
     Closed←∅;
     Frontier ← <InitialState>;
     CurrentState ← First(Frontier);
            while CurrentState ∉ FinalStates do
                   Frontier ←delete(CurrentState,Frontier);
                   if CurrentState ∉ Closed then
                     ChildrenStates ← Expand(CurrentState);
                     Frontier ←ChildrenStates ^ Frontier;
                      Closed ← Closed{CurrentState};
                   endif;
                   if Frontier=∅ then return fail;
                   CurrentState ← First(Frontier);
            endwhile;
     return success;
     end.
```

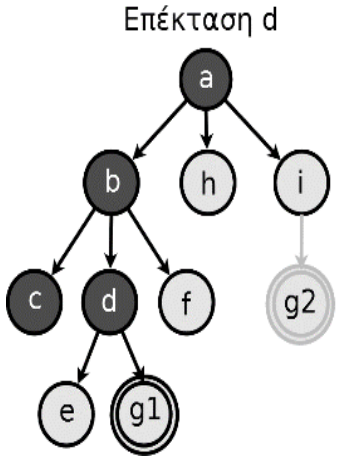
Αναζήτηση Πρώτα σε Βάθος σε Χώρο Αναζήτησης με αρχική κατάσταση *a* και τελικές *g1* και *g2*.

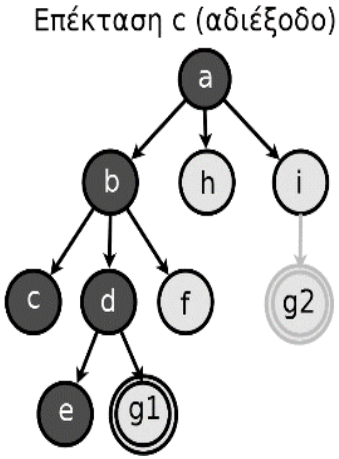


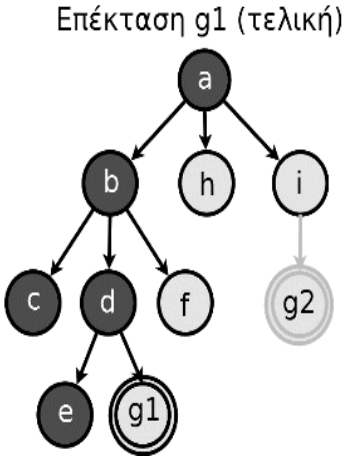


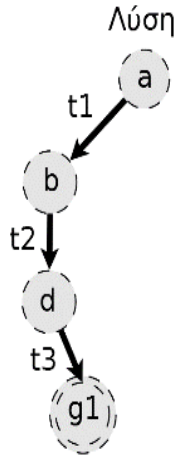




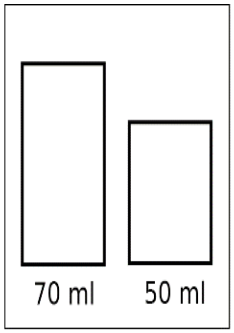




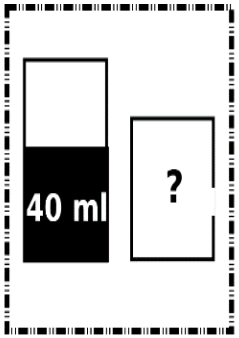




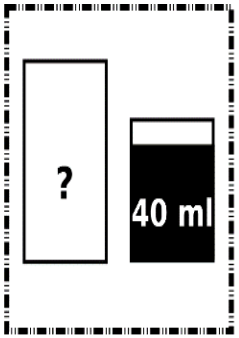
Αρχική και τελικές καταστάσεις του προβλήματος των ποτηριών.



Αρχική Κατάσταση

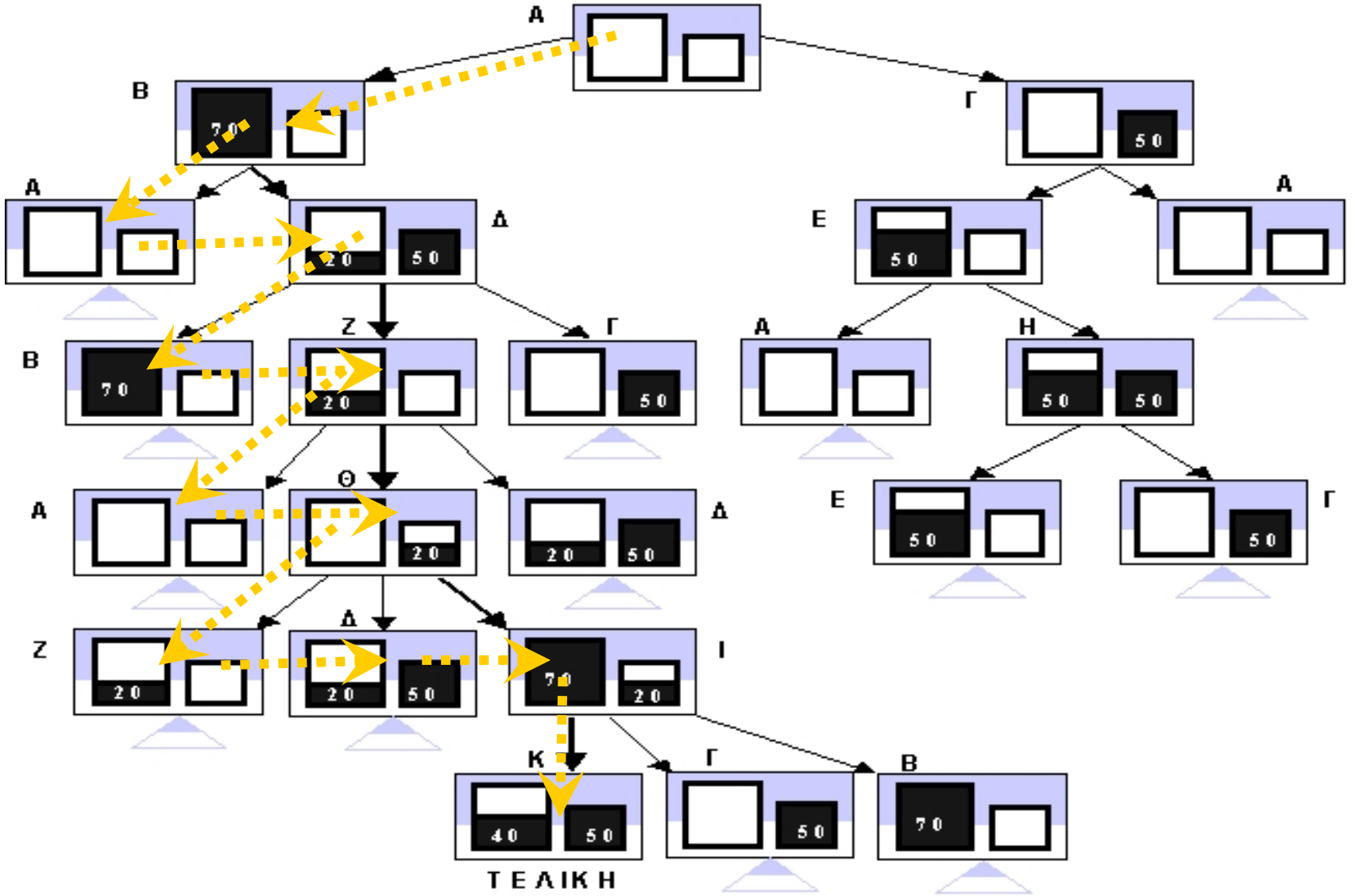


Τελική Κατάσταση



Τελική Κατάσταση

Αναζήτηση πρώτα σε βάθος (DFS)



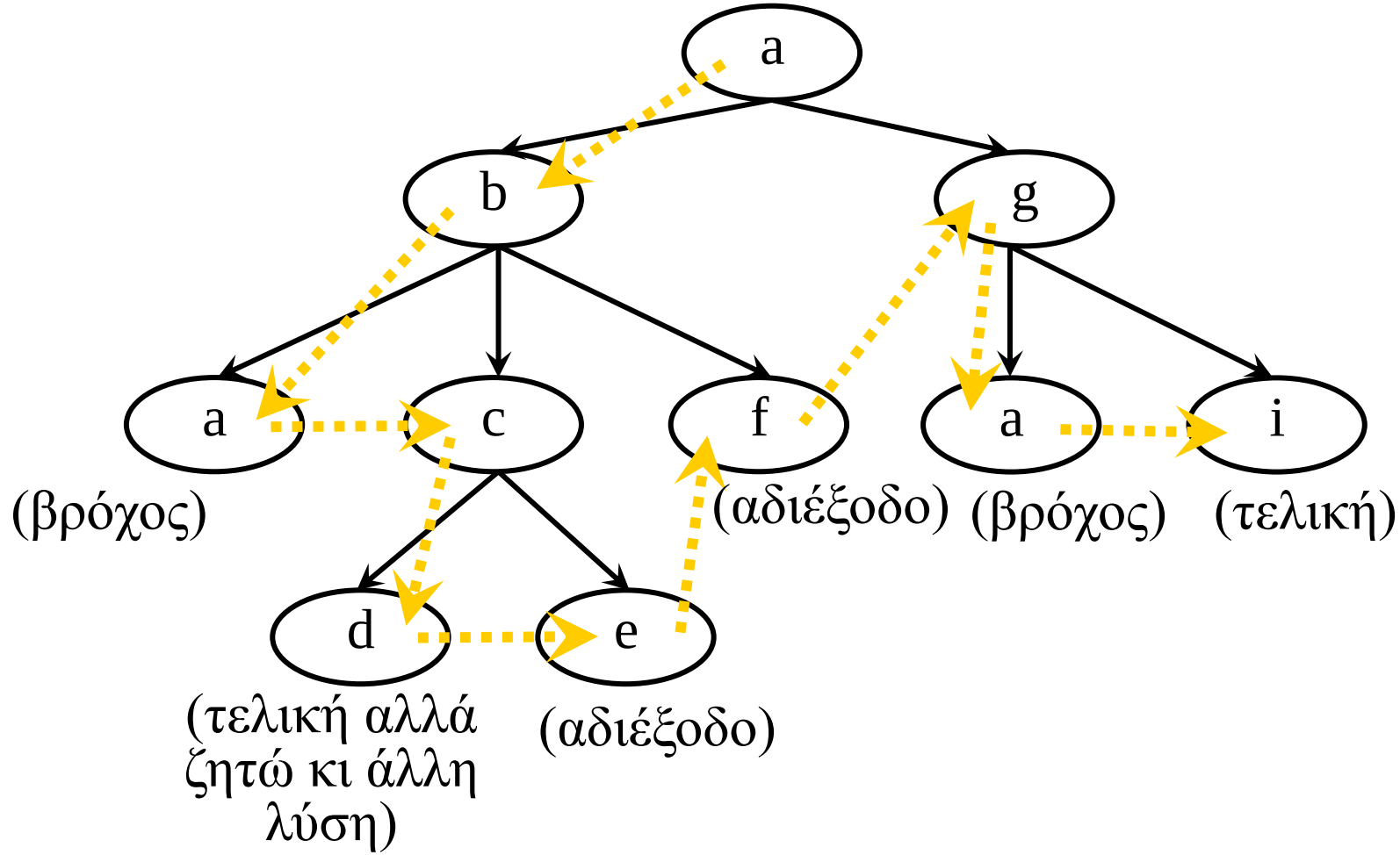
Παράδειγμα αναζήτησης (DFS)

Μέτωπο αναζήτησης	Κλειστό σύνολο	Κατάσταση	Παιδιά
(Α)	{}	Α	Β, Γ
(Β, Γ)	{Α}	Β	Α, Δ
(Α, Δ, Γ)	{Α, Β}	Α	(βρόχος)
(Δ, Γ)	{Α, Β}	Δ	Β, Ζ, Γ
(Β, Ζ, Γ, Γ)	{Α, Β, Δ}	Β	(βρόχος)
(Ζ, Γ, Γ)	{Α, Β, Δ}	Ζ	Α, Θ, Δ
(Α, Θ, Δ, Γ, Γ)	{Α, Β, Δ, Ζ}	Α	(βρόχος)
(Θ, Δ, Γ, Γ)	{Α, Β, Δ, Ζ}	Θ	Ζ, Δ, Ι
(Ζ, Δ, Ι, Δ, Γ, Γ)	{Α, Β, Δ, Ζ, Θ}	Ζ	(βρόχος)
(Δ, Ι, Δ, Γ, Γ)	{Α, Β, Δ, Ζ, Θ}	Δ	(βρόχος)
(Ι, Δ, Γ, Γ)	{Α, Β, Δ, Ζ, Θ}	Ι	Κ, Γ, Β
(Κ, Γ, Β, Δ, Γ, Γ)	{Α, Β, Δ, Ζ, Θ, Ι}	Κ	(τελική)

Χαρακτηριστικά του DFS

- Το μέτωπο της αναζήτησης είναι μια δομή στοίβας (Stack LIFO, Last In First Out)
- Η εξέταση αμέσως προηγουμένων (χρονικά) καταστάσεων ονομάζεται χρονική οπισθοδρόμηση (chronological backtracking).
- Πλεονεκτήματα:
  - Έχει μικρές απαιτήσεις σε χώρο διότι το μέτωπο της αναζήτησης δε μεγαλώνει πάρα πολύ.
- Μειονεκτήματα:
  - Δεν εγγυάται ότι η πρώτη λύση που θα βρεθεί είναι η βέλτιστη (μονοπάτι με το μικρότερο μήκος ή με μικρότερο κόστος).
  - Εν γένει θεωρείται ατελής (αν δεν υπάρχει έλεγχος βρόχων ή αν ο χώρος αναζήτησης είναι μη πεπερα-σμένος)

Οπισθοδρόμηση στον DFS



Οπισθοδρόμηση στον DFS

Μέτωπο	Κλειστό σύνολο	Κατ/ση	Παιδιά
(a)	{}	a	b, g
(b, g)	{a}	b	a, c, f
(a, c, f, g)	{a, b}	a	(βρόχος)
(c, f, g)	{a, b}	c	d, e
(d, e, f, g)	{a, b, c}	d	(τελική)
(e, f, g)	{a, b, c, d}	e	(δεν υπάρχουν)
(f, g)	{a, b, c, d, e}	f	(δεν υπάρχουν)
(g)	{a, b, c, d, e, f}	g	a, i
(a, i)	(a, b, c, d, e, f, g}	a	(βρόχος)
(i)	(a, b, c, d, e, f, g}	i	(τελική)

Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

Αναζήτηση πρώτα σε πλάτος (Breadth-First Search BFS)

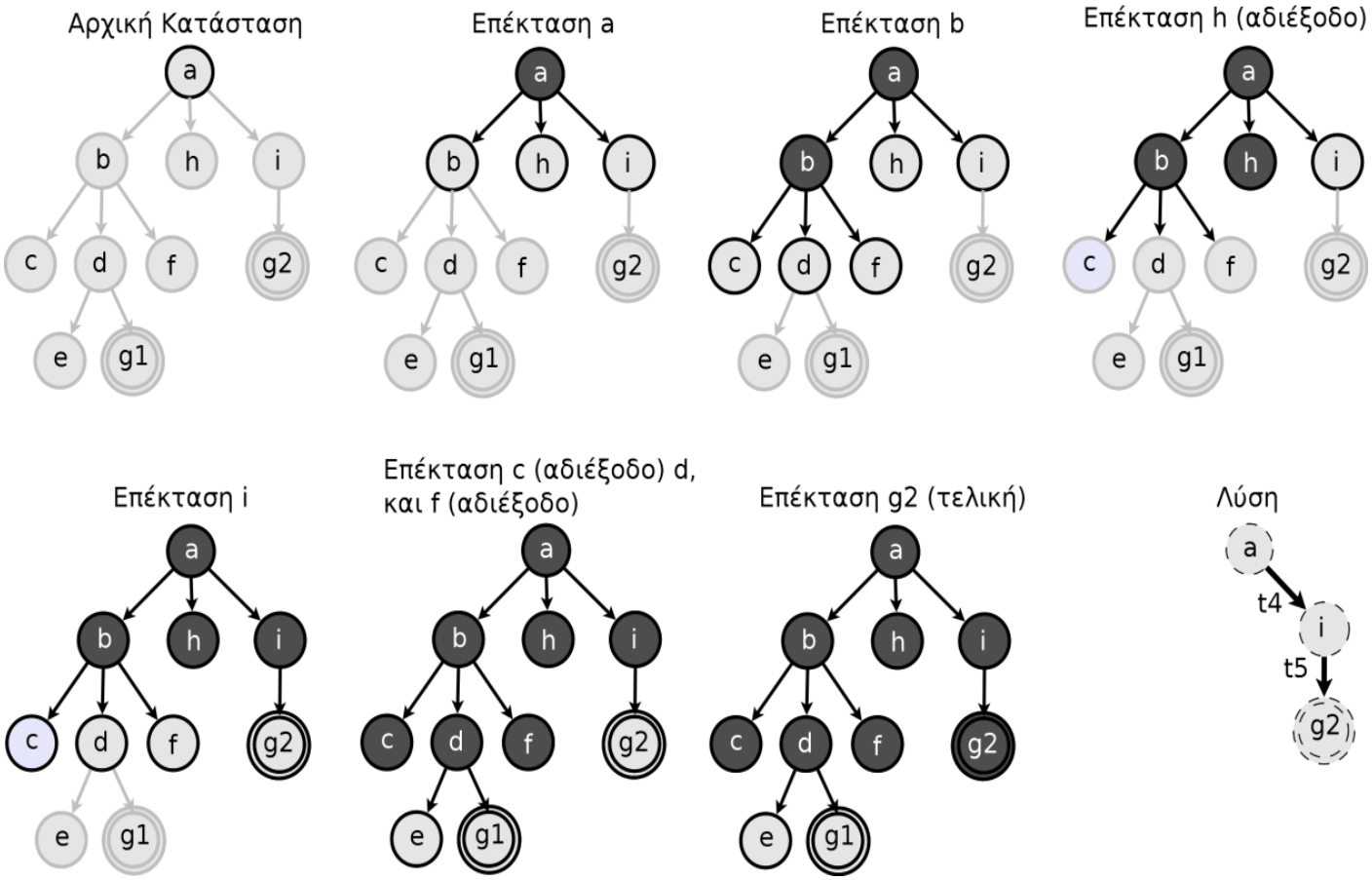
Ο αλγόριθμος *αναζήτησης πρώτα σε πλάτος* εξετάζει πρώτα όλες τις καταστάσεις που βρίσκονται στο ίδιο βάθος και μετά συνεχίζει στην επέκταση καταστάσεων στο αμέσως επόμενο επίπεδο.

1. Βάλε την αρχική κατάσταση στο μέτωπο της αναζήτησης.
2. Αν το μέτωπο της αναζήτησης είναι κενό τότε σταμάτησε.
3. Βγάλε την πρώτη κατάσταση από το μέτωπο της αναζήτησης.
4. Αν είναι η κατάσταση μέλος του κλειστού συνόλου τότε πήγαινε στο βήμα 2.
5. Αν η κατάσταση είναι μία τελική τότε ανέφερε τη λύση.
6. Αν θέλεις και άλλες λύσεις πήγαινε στο βήμα 2. Αλλιώς σταμάτησε.
7. Εφάρμοσε τους τελεστές μεταφοράς για να βρεις τις καταστάσεις- παιδιά.
8. Βάλε τις **καταστάσεις-παιδιά στο τέλος του μετώπου** της αναζήτησης.
9. Βάλε την κατάσταση-γονέα στο κλειστό σύνολο.
10. Πήγαινε στο βήμα 2.

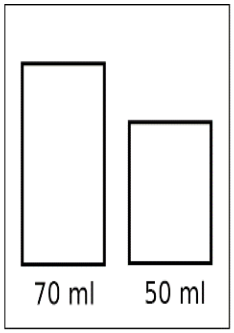
Αναζήτηση πρώτα σε πλάτος (Breadth-First Search BFS)

```
algorithm bfs(InitialState, FinalStates)
   begin
        Closed ← ∅;
        Frontier ← <InitialState>;
        CurrentState ← First(Frontier);
               while CurrentState ∉ FinalStates do
                      Frontier ← delete(CurrentState,Frontier);
                        if CurrentState ∉ Closed
                       ChildrenStates ← Expand(CurrentState);
                        Frontier ← Frontier ^ ChildrenStates;
                        Closed ← Closed{CurrentState};
                             endif;
                      if Frontier= ∅ then return fail;
                      CurrentState ← First(Frontier);
               endwhile;
               return success;
        end.
```

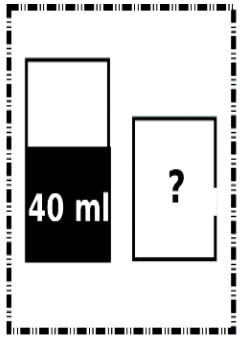
Αναζήτηση Πρώτα σε Πλάτος σε Χώρο Αναζήτησης με αρχική κατάσταση *a* και τελικές *g1* και *g2*.



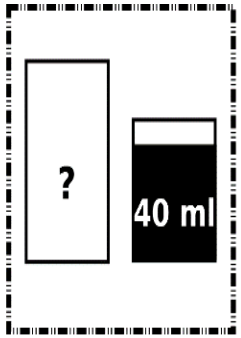
Αρχική και τελικές καταστάσεις του προβλήματος των ποτηριών.



Αρχική Κατάσταση

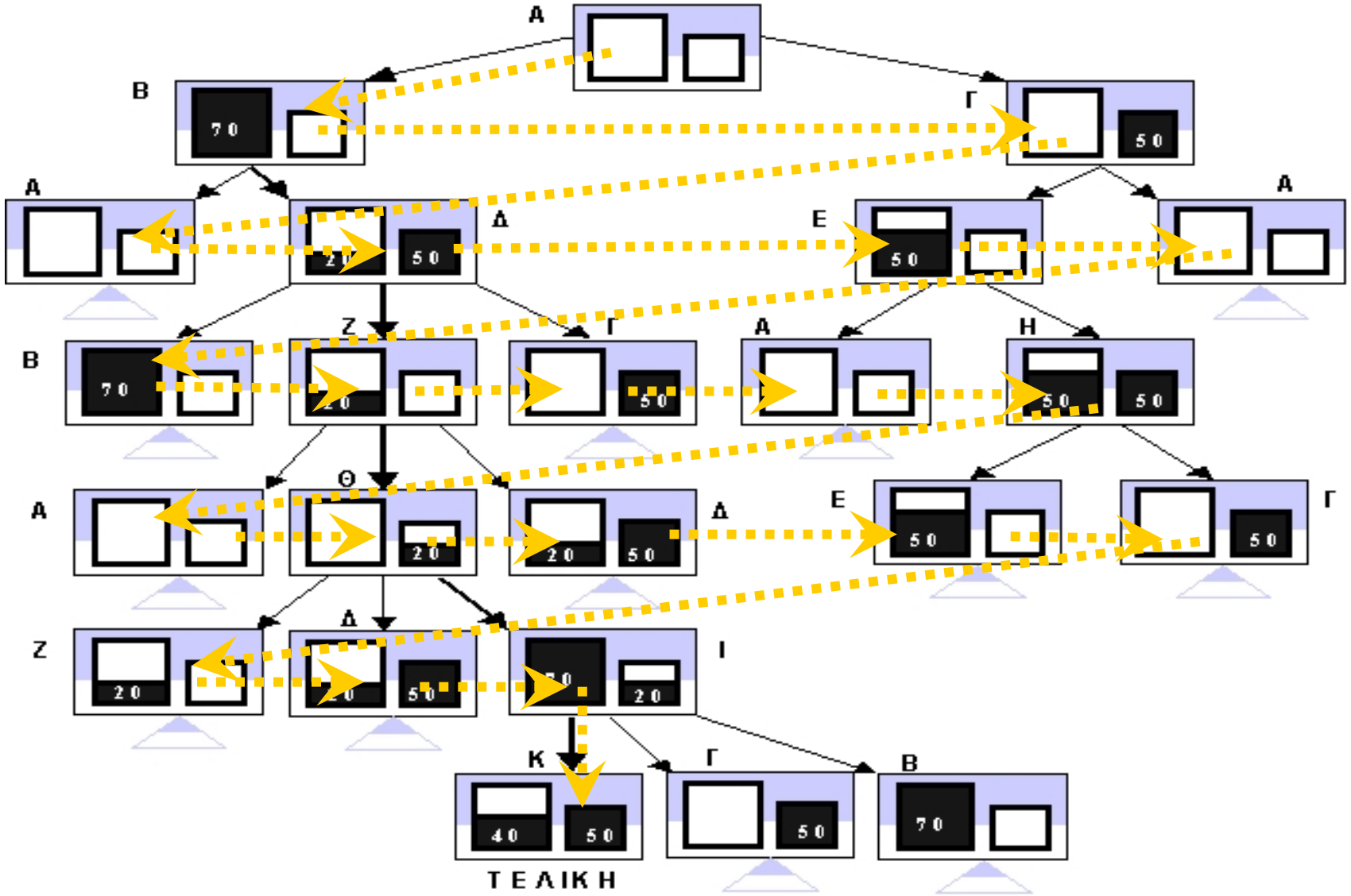


Τελική Κατάσταση



Τελική Κατάσταση

Αναζήτηση πρώτα σε πλάτος (BFS)



Παράδειγμα αναζήτησης BFS

Μέτωπο αναζήτησης	Κλειστό σύνολο	Κατάσταση	Παιδιά
(Α)	{}	Α	Β, Γ
(Β, Γ)	{Α}	Β	Α, Δ
(Γ, Α, Δ)	{Α, Β}	Γ	Ε, Α
(Α, Δ, Ε, Α)	{Α, Β, Γ}	Α	(βρόχος)
(Δ, Ε, Α)	{Α, Β, Γ}	Δ	Β, Ζ, Γ
(Ε, Α, Β, Ζ, Γ)	{Α, Β, Γ, Δ}	Ε	Α, Η
(Α, Β, Ζ, Γ, Α, Η)	{Α, Β, Γ, Δ, Ε}	Α	(βρόχος)
(Β, Ζ, Γ, Α, Η)	{Α, Β, Γ, Δ, Ε}	Β	(βρόχος)
(Ζ, Γ, Α, Η)	{Α, Β, Γ, Δ, Ε}	Ζ	Α, Θ, Δ
(Γ, Α, Η, Α, Θ, Δ)	{Α, Β, Γ, Δ, Ε, Ζ}	Γ	(βρόχος)
(Α, Η, Α, Θ, Δ)	{Α, Β, Γ, Δ, Ε, Ζ}	Α	(βρόχος)
(Η, Α, Θ, Δ)	{Α, Β, Γ, Δ, Ε, Ζ}		

Παράδειγμα BFS – συνέχεια

Μέτωπο	Κλειστό σύνολο	Κατ/ση	Παιδιά
(Η, Α, Θ, Δ)	{Α, Β, Γ, Δ, Ε, Ζ}	Η	Ε, Γ
(Α, Θ, Δ, Ε, Γ)	{Α, Β, Γ, Δ, Ε, Ζ, Η}	Α	(βρόχος)
(Θ, Δ, Ε, Γ)	{Α, Β, Γ, Δ, Ε, Ζ, Η}	Θ	Ζ, Δ, Ι
(Δ, Ε, Γ, Ζ, Δ, Ι)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ}	Δ	(βρόχος)
(Ε, Γ, Ζ, Δ, Ι)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ}	Ε	(βρόχος)
(Γ, Ζ, Δ, Ι)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ}	Γ	(βρόχος)
(Ζ, Δ, Ι)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ}	Ζ	(βρόχος)
(Δ, Ι)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ}	Δ	(βρόχος)
(Ι)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ}	Ι	Κ, Γ, Β
(Κ, Γ, Β)	{Α, Β, Γ, Δ, Ε, Ζ, Η, Θ, Ι}	Κ	(τελική)

Χαρακτηριστικά BFS

- Το μέτωπο της αναζήτησης είναι μια δομή ουράς (Queue FIFO, δηλαδή First In First Out).
- Πλεονεκτήματα:
  - Βρίσκει πάντα την καλύτερη λύση (μικρότερη σε μήκος).
  - Είναι πλήρης.
- Μειονεκτήματα:
  - Το μέτωπο της αναζήτησης μεγαλώνει πολύ σε μέγεθος.

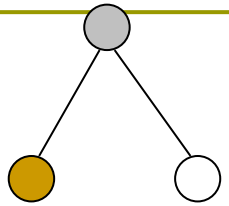
Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

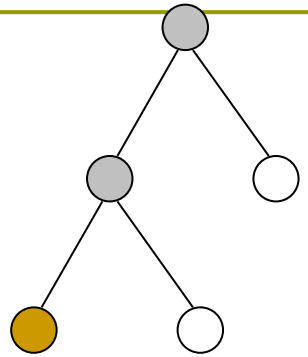
DFS – Graphical Example



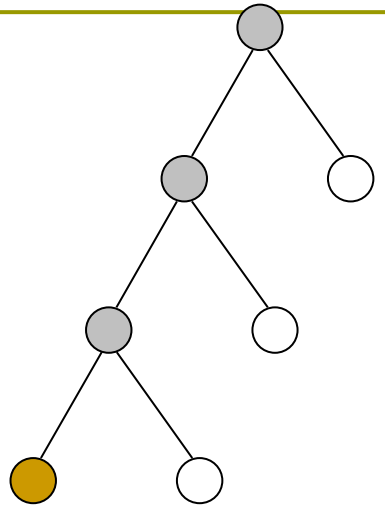
DFS – Graphical Example



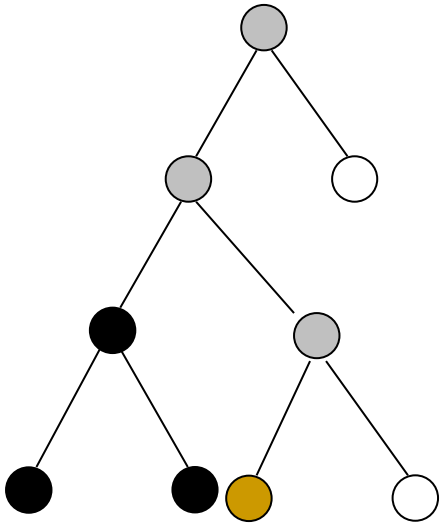
DFS – Graphical Example



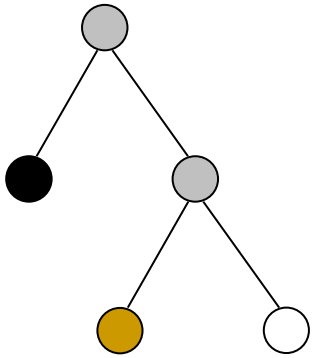
DFS – Graphical Example



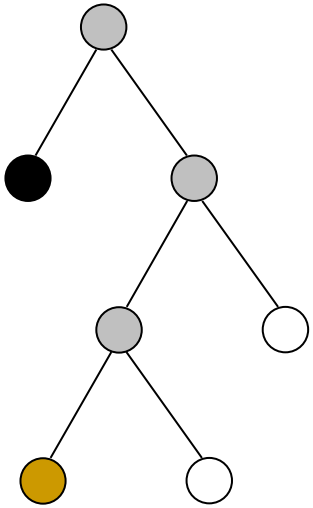
DFS – Graphical Example



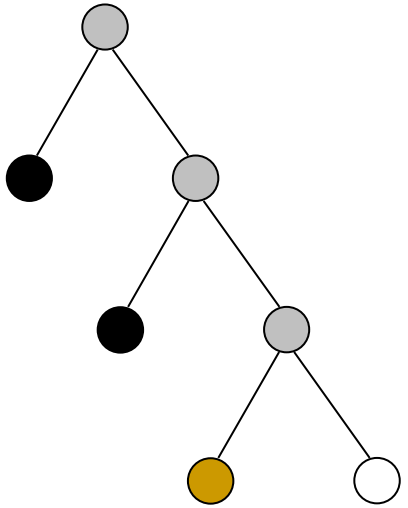
DFS – Graphical Example



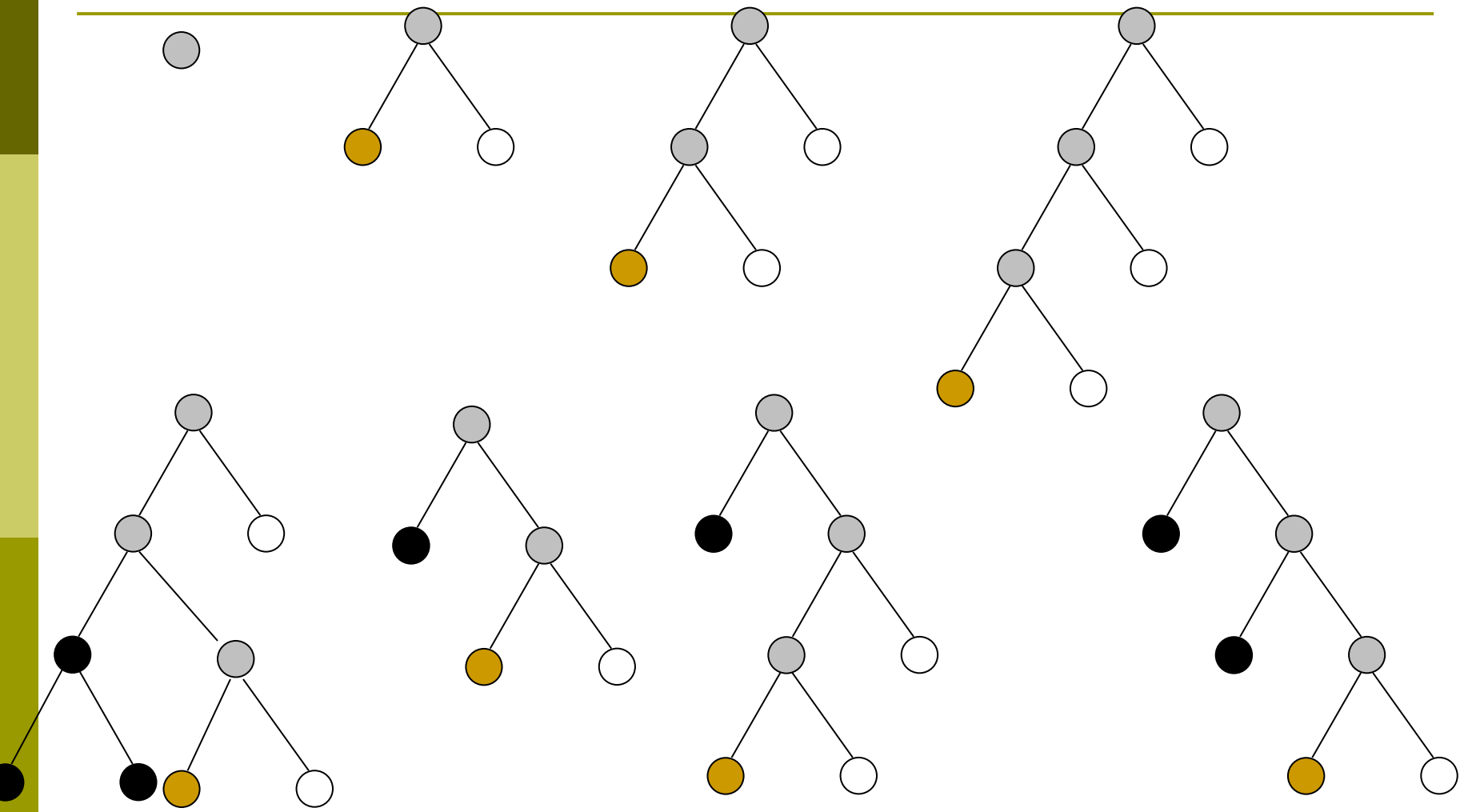
DFS – Graphical Example



DFS – Graphical Example



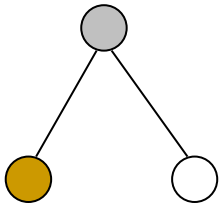
DFS – Graphical Example



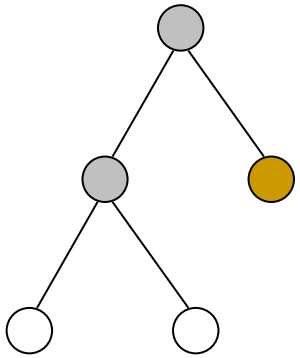
BFS – Graphical Example



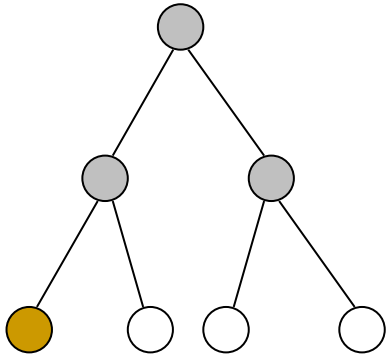
BFS – Graphical Example



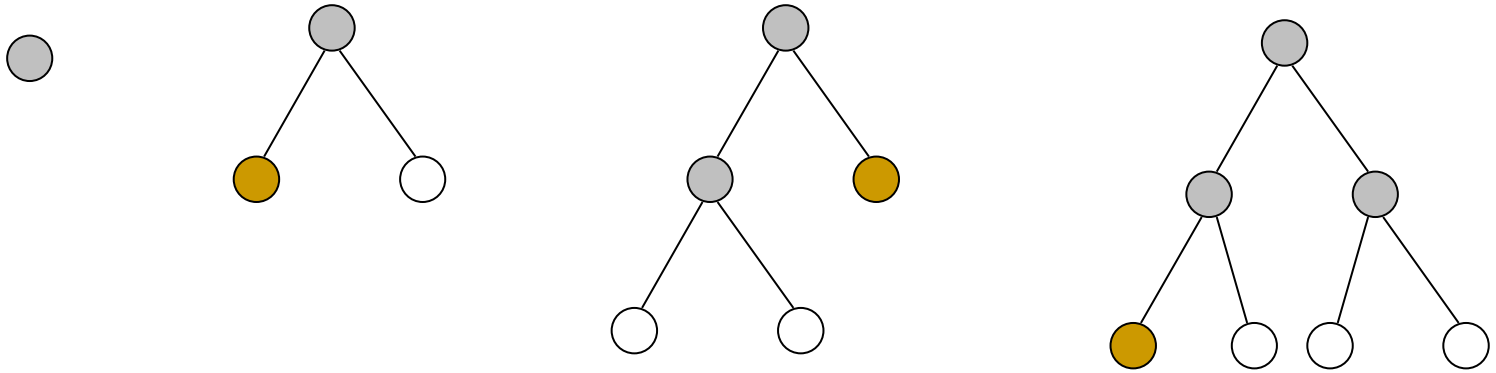
BFS – Graphical Example



BFS – Graphical Example



BFS – Graphical Example



Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

Επαναληπτική εμβάθυνση
Iterative deepening (ID)

- Ο αλγόριθμος επαναληπτικής εκβάθυνσης συνδυάζει τους DFS και BFS ως εξής:
  - Ορίζει ένα αρχικό βάθος αναζήτησης, έστω d.
  - Εκτελεί τον DFS σε όλο το χώρο αναζήτησης μέχρι κάποιο συγκεκριμένο βάθος.
  - Εάν δεν βρεθεί λύση, αυξάνει το d και ξαναεκτελεί τον DFS εξαρχής και μέχρι το νέο βάθος.

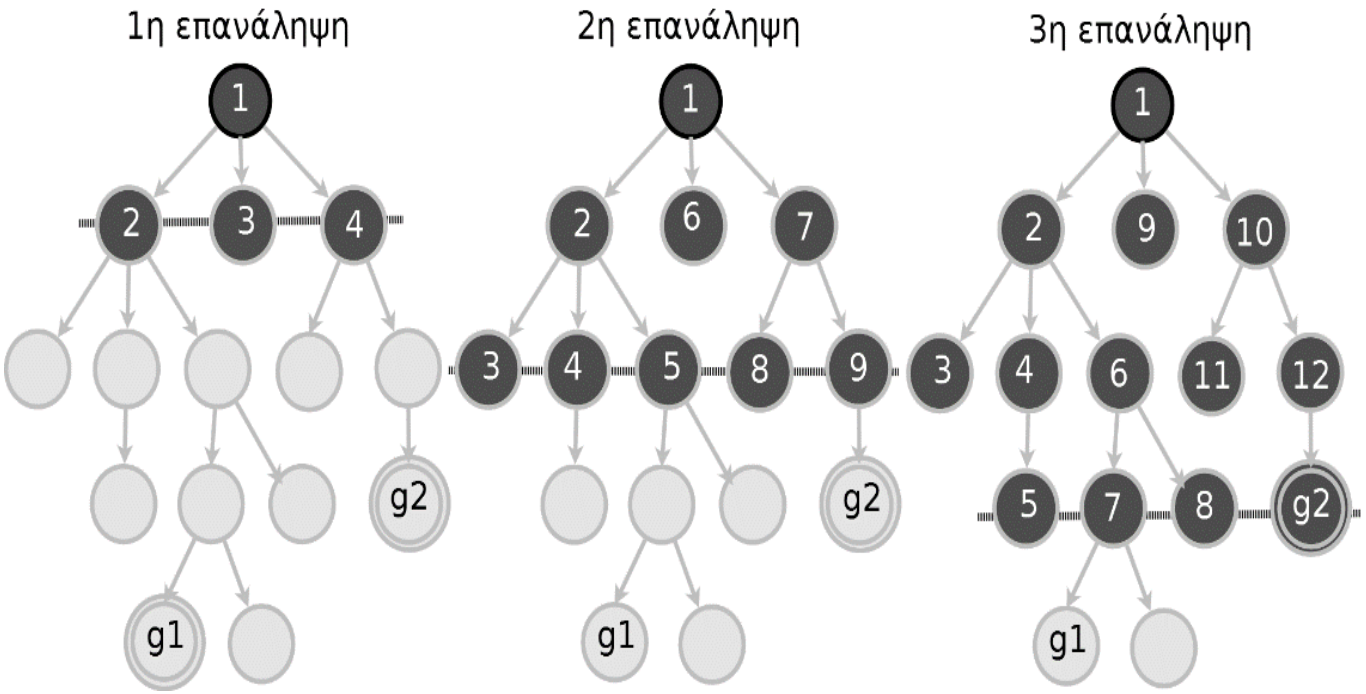
Επαναληπτική εμβάθυνση (ID)

1. Όρισε το **αρχικό βάθος** αναζήτησης (π.χ. 1).
2. Εφάρμοσε τον **DFS** (χωρίς κλειστό σύνολο) μέχρι το βάθος αναζήτησης.
3. Αν βρέθηκε λύση, σταμάτησε.
4. **Αύξησε το βάθος** αναζήτησης (κατά 1 ή περισσότερο).
5. Πήγαινε στο βήμα 2.

Επαναληπτική εμβάθυνση (ID)

```
algorithm id(InitialState, FinalStates)
begin
  depth  ← 1
 while solution is not found do
  bounded_dfs(InitialState,FinalStates,depth);
                depth  ←  depth+1
       endwhile;
end.
```

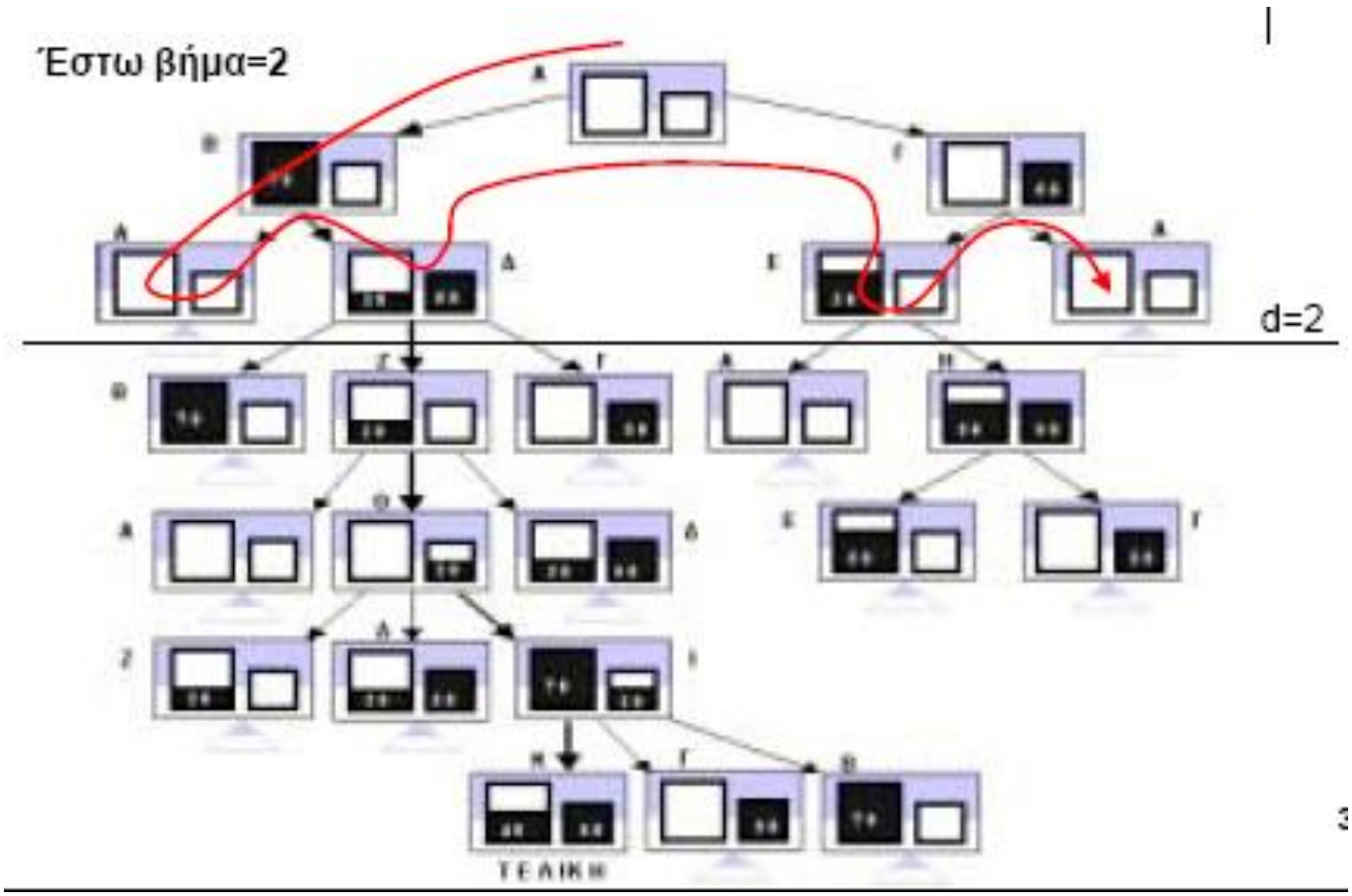
Αναζήτηση με επαναληπτικής εκβάθυνση σε Χώρο Αναζήτησης με τελικές καταστάσεις g_1 και g_2.



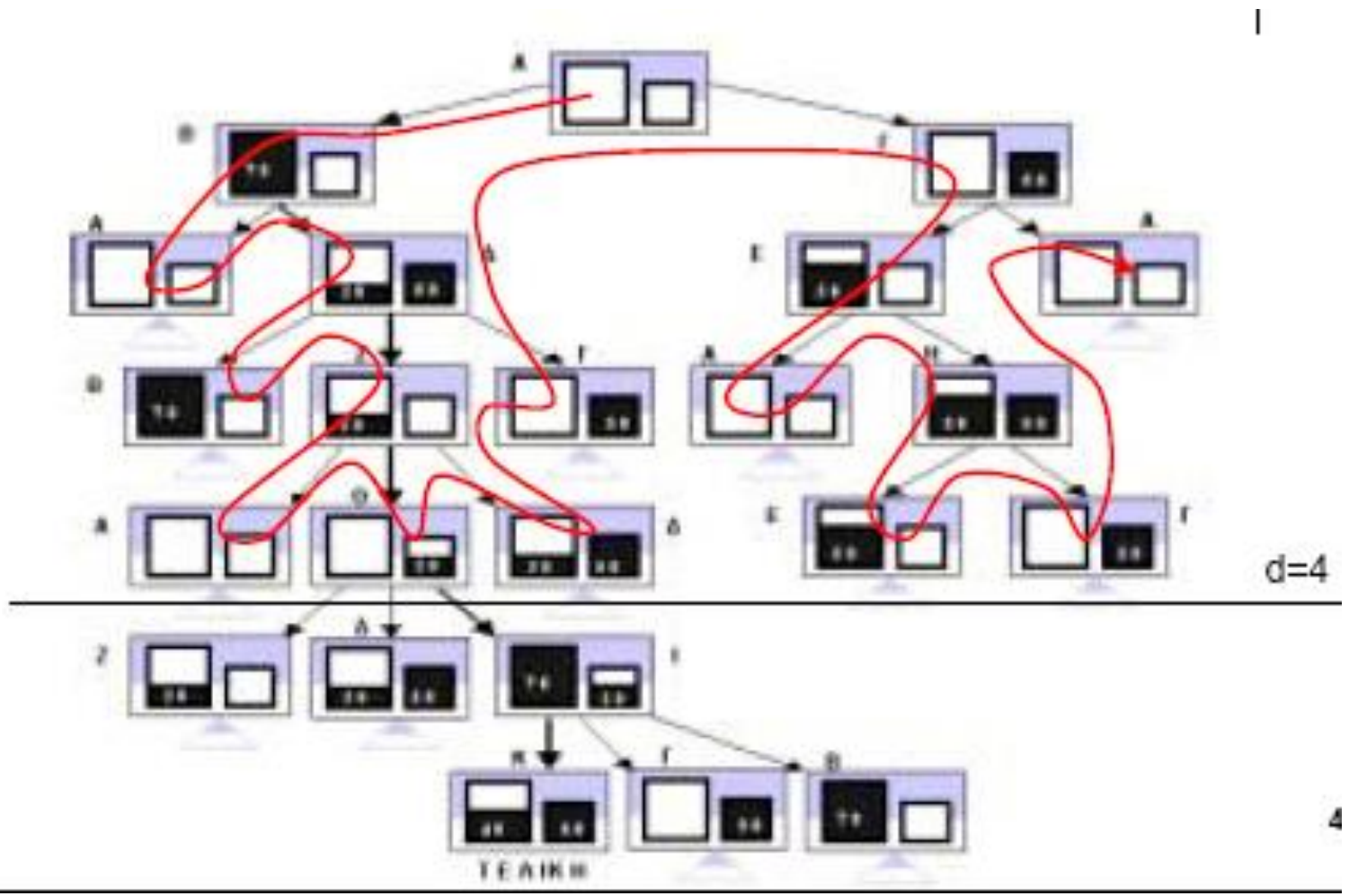
Επαναληπτική εμβάθυνση (ID)

- Αποτελεί **συνδυασμό DFS και BFS**.
- Για βήμα αύξησης βάθους 1, βρίσκει πρώτα τη συντομότερη λύση, όπως ο BFS (**αποδεκτός**).
- **Πλήρης** σε πεπερασμένους χώρους αναζήτησης.
- **Μικρές απαιτήσεις σε μνήμη**, όπως ο DFS.
- Σε κάθε επανάληψη **δε θυμάται τίποτα** από τις προηγούμενες!

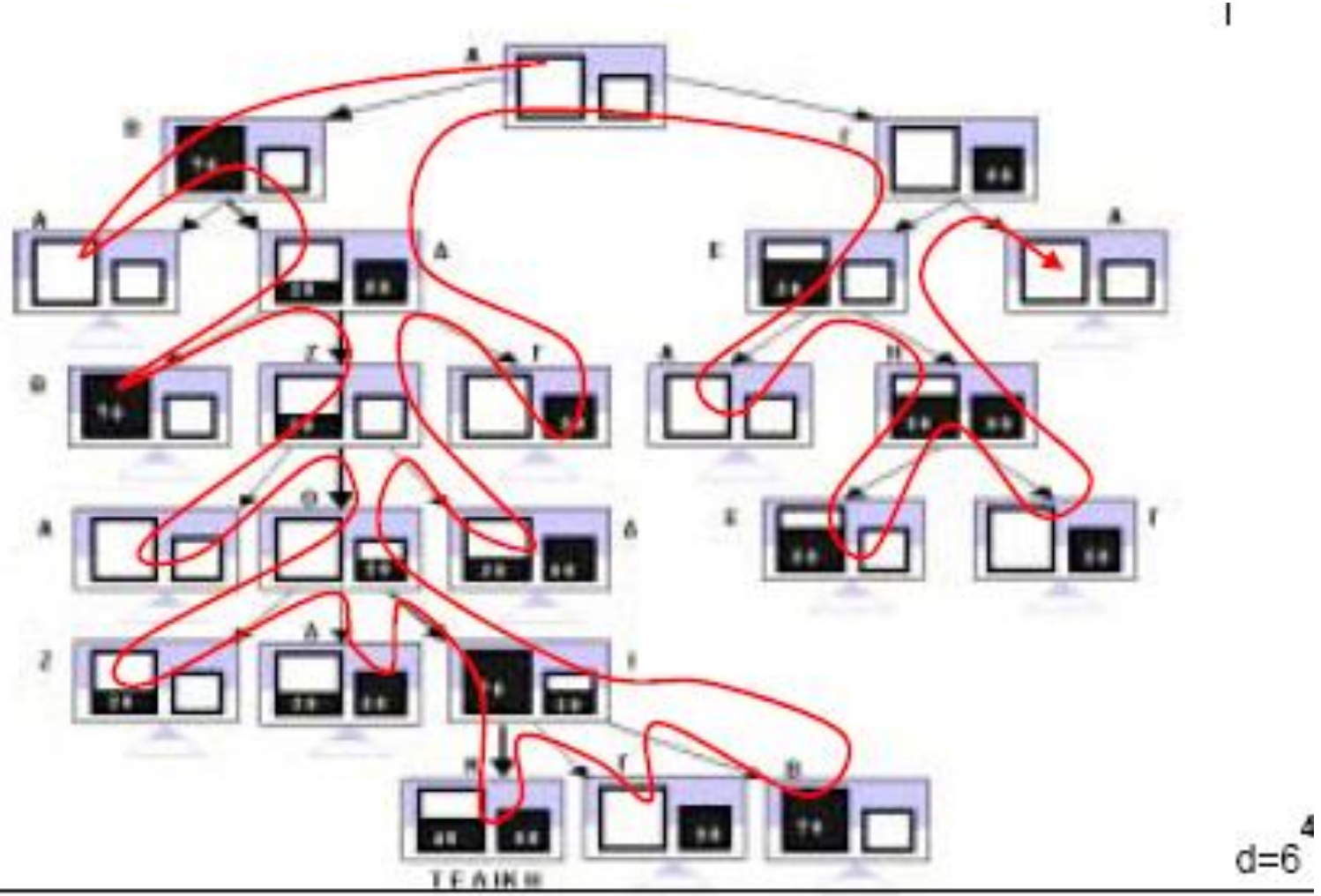
Επαναληπτική εμβάθυνση (ID)



Επαναληπτική εμβάθυνση (ID)



Επαναληπτική εμβάθυνση (ID)



Παρατηρήσεις

- Ο αλγόριθμος ID έχει την ίδια πολυπλοκότητα σε χώρο και χρόνο με DFS και BFS
- Παράδειγμα, για Δένδρο Αναζήτησης με μέσο παράγοντα διακλάδωσης 10, εφαρμόζοντας τον DFS σε βάθος 9, οι κατάστάσεις που θα επεκταθούν είναι 10^9 . Εάν αυξηθεί το βάθος κατά 1, τότε για βάθος 10 εξετάζονται 10^{10} καταστάσεις. Άρα αύξηση μόνο 10%

Παρατηρήσεις

- Μειονεκτήματα:
  - Όταν ο αλγόριθμος ID ξαναρχίσει με διαφορετικό βάθος, δεν θυμάται τίποτα από την προηγούμενη αναζήτηση.
- Πλεονεκτήματα:
  - Είναι πλήρης.
  - Έχει μικρές απαιτήσεις σε μνήμη.
  - Δεν χάνεται σε κλαδιά άπειρου μήκους.
  - Αν το βάθος αυξάνεται κατά 1 σε κάθε κύκλο και ο ID βρει λύση, τότε αυτή η λύση θα είναι η καλύτερη.
  - Αν το βάθος αυξάνεται κατά d σε κάθε βήμα, τότε η πρώτη λύση που θα βρεθεί θα είναι το πολύ κατά d-1 μεγαλύτερη της βέλτιστης.

Επίλυση Προβλημάτων

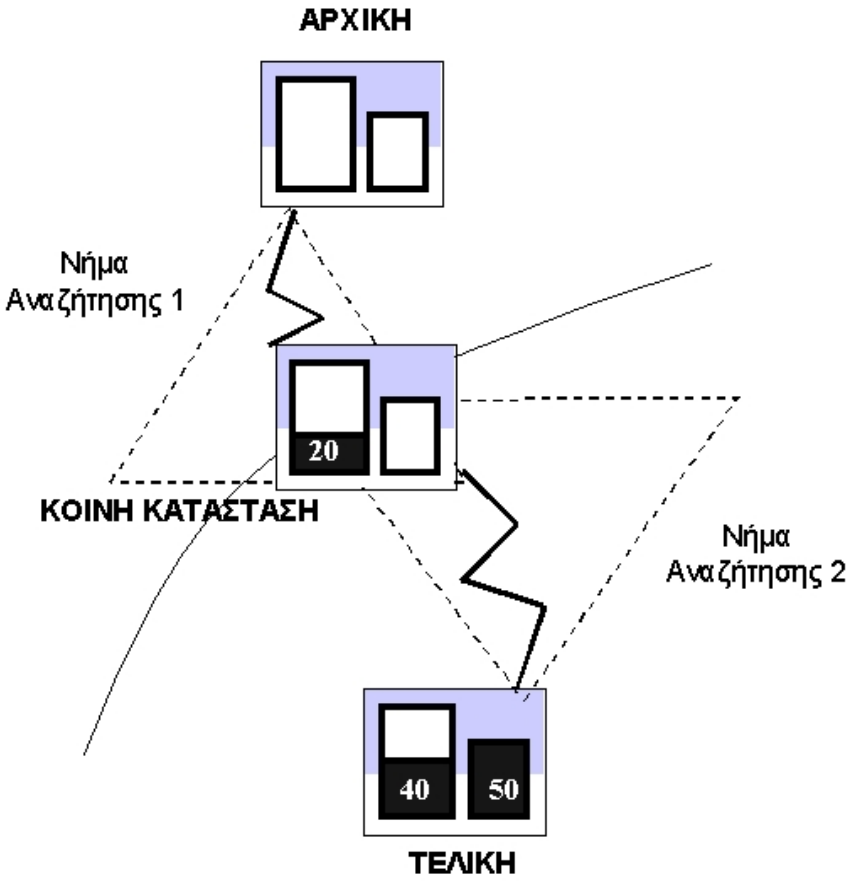
- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

Αναζήτηση διπλής κατεύθυνσης (Bidirectional Search, BiS)

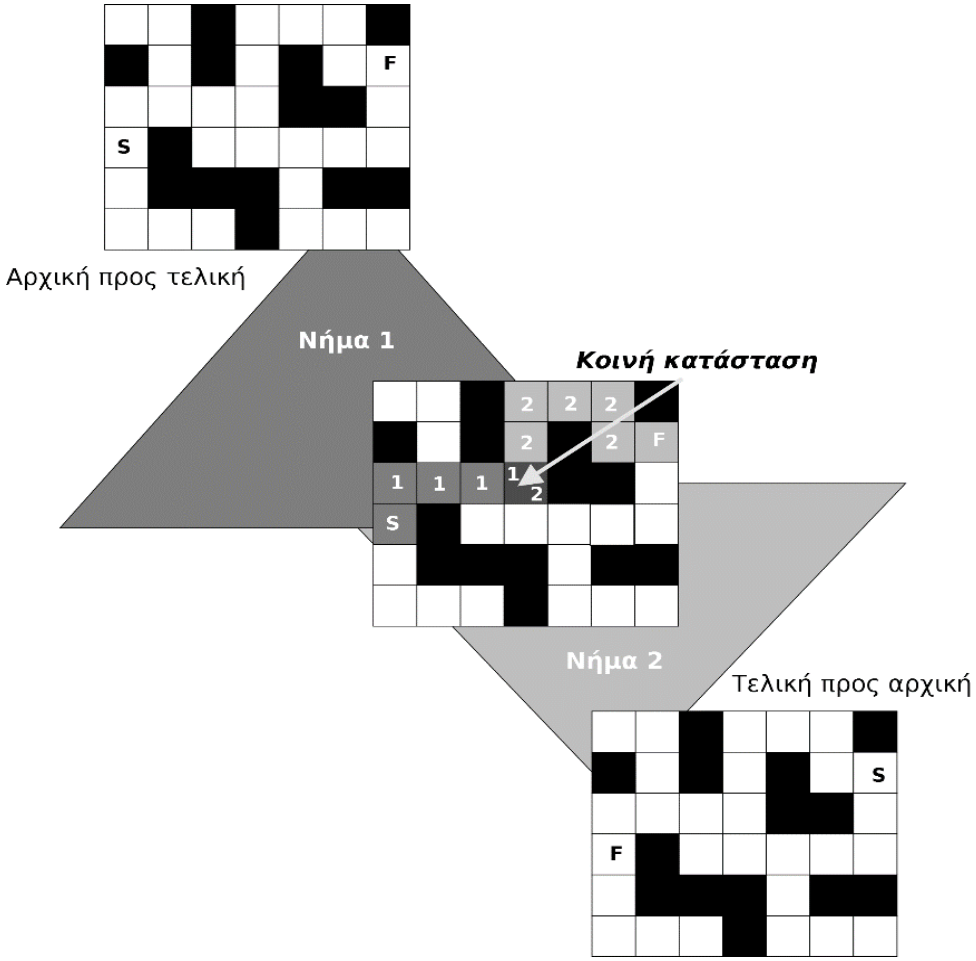
- Γενική ιδέα:
  - Ξεκινούμε να ψάχνουμε ταυτόχρονα από την αρχική προς την τελική κατάσταση, αλλά και από την τελική προς την αρχική.
    - Οι δύο κατευθύνσεις της αναζήτησης μπορούν να εκτελεστούν παράλληλα.
  - Όταν οι δύο κατευθύνσεις αναζήτησης συναντήσουν την ίδια κατάσταση, ο αλγόριθμος τερματίζει.
  - " Δεν μας ενδιαφέρει ποιος αλγόριθμος αναζήτησης χρησιμοποιείται σε κάθε κατεύθυνση (μπορεί να είναι και διαφορετικοί).
- Προϋποθέσεις κάτω από τις οποίες μπορεί να εφαρμοστεί:
  - Οι τελεστές μετάβασης είναι αντιστρέψιμοι (reversible), και
  - Είναι πλήρως γνωστή η τελική κατάσταση.
- Μειονεκτήματα
  - Υπάρχει επιπλέον κόστος που οφείλεται στην επικοινωνία μεταξύ των δύο αναζητήσεων.

Αναζήτηση διπλής κατεύθυνσης

- Δυνατότητα **παράλληλης αναζήτησης**.
- Οι τελεστές μετάβασης πρέπει να είναι **αντιστρέψιμοι**.
- Η τελική κατάσταση πρέπει να είναι **πλήρως γνωστή**.
- **Κόστος επικοινωνίας** μεταξύ των δύο κατευθύνσεων.



Αναζήτηση διπλής κατεύθυνσης στο πρόβλημα εύρεσης διαδρομής σε λαβύρινθο.



Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (BiS)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

Επέκταση και Οριοθέτηση (B &B)

- Κατά την αναζήτηση ΔΕΝ εξετάζονται οι καταστάσεις που είναι βέβαιο ότι ΔΕΝ θα οδηγήσουν στην καλύτερη λύση. ΚΛΑΔΕΜΑ του χώρου Αναζήτησης.
- Εισάγεται η έννοια του μερικού κόστους για να εφαρμόσουμε το κλάδεμα.
- Μερικό κόστος ορίζεται το κόστος εφαρμογής τελεστών μέχρι μια συγκεκριμένη μη-τελική κατάσταση.
- Αν μια κατάσταση έχει **μερικό κόστος μεγαλύτερο** από εκείνο μιας πλήρους λύσης, τότε η κατάσταση είναι βέβαιο ότι δεν θα οδηγήσει σε καλύτερη λύση. **Οπότε κλαδεύεται.**

Επέκταση και Οριοθέτηση (B &B) Παράδειγμα



Σχηματική αναπαράσταση της αρχικής κατάστασης του του κόσμου της οικογένειας που περνάει τη γέφυρα

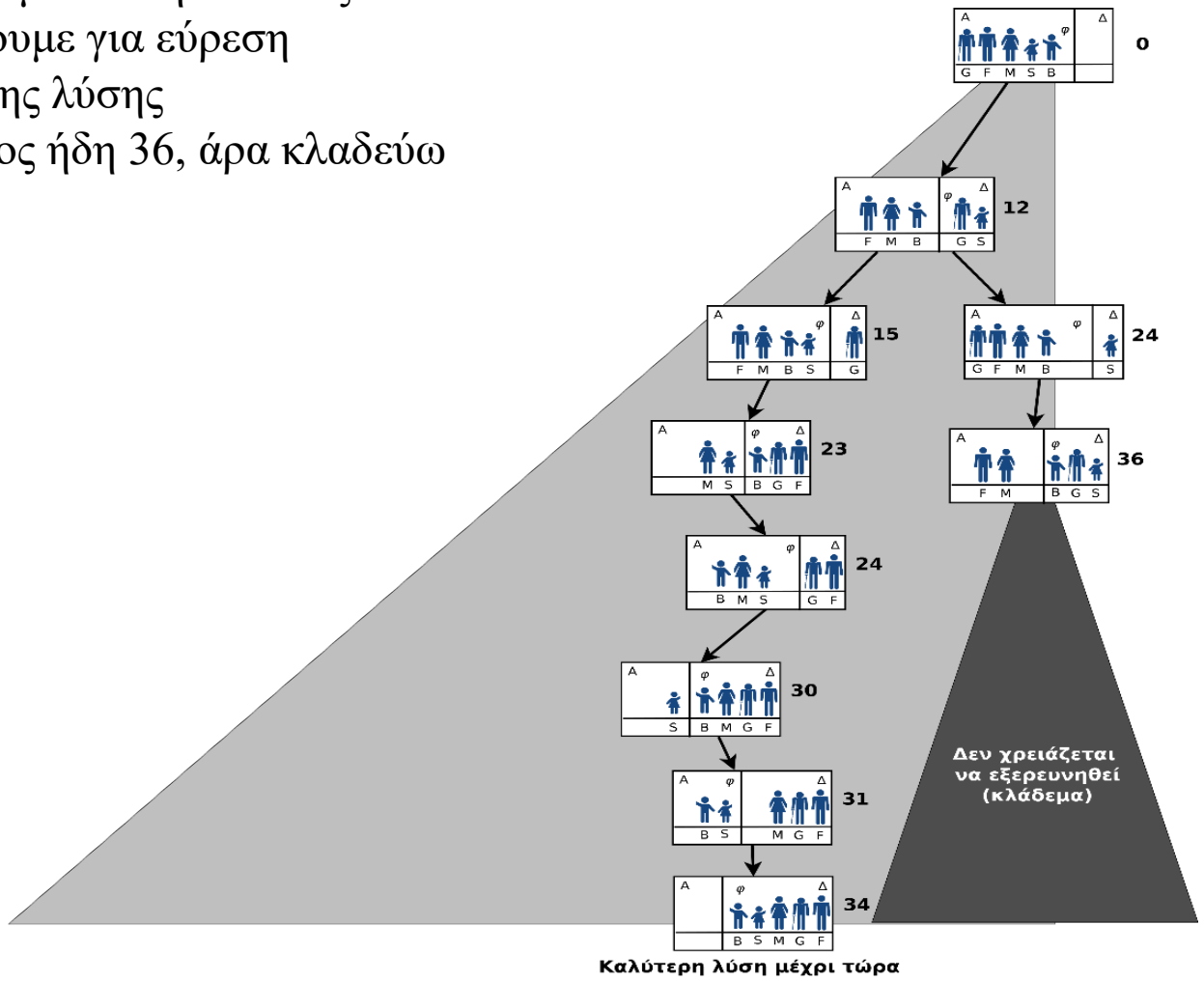
G: 12min
F: 8 min
M: 6 min
S: 3 min
B: 1 min

Φακός: 30min

Διαδοχικές καταστάσεις κατά την εξέλιξη
του κόσμου της οικογένειας που περνάει την γέφυρα.

Επέκταση και Οριοθέτηση (B &B)

- Βρέθηκε μια λύση κόστους 34.
- Συνεχίζουμε για εύρεση καλύτερης λύσης
- Το κόστος ήδη 36, άρα κλαδεύω



Επέκταση και Οριοθέτηση (B &B)

1. Βάλε την αρχική κατάσταση στο μέτωπο της αναζήτησης.
2. Αρχική τιμή της καλύτερης λύσης είναι το +οο (όριο).
3. Αν το μέτωπο της αναζήτησης είναι κενό, τότε σταμάτησε. Η καλύτερη μέχρι τώρα λύση είναι και η βέλτιστη.
4. Βγάλε την πρώτη σε σειρά κατάσταση από το μέτωπο της αναζήτησης.
5. Αν η κατάσταση ανήκει στο κλειστό σύνολο, τότε πήγαινε στο βήμα 3.
6. Αν η κατάσταση είναι τελική, τότε ανανέωσε τη λύση ως την καλύτερη μέχρι τώρα και ανανέωσε την τιμή του ορίου με την τιμή που αντιστοιχεί στην τελική κατάσταση. Πήγαινε στο βήμα 3.
7. Εφάρμοσε τους τελεστές μετάβασης για να παράγεις τις καταστάσεις- παιδιά και την τιμή που αντιστοιχεί σε αυτές.
8. Βάλε τις καταστάσεις-παιδιά, των οποίων η τιμή δεν υπερβαίνει το όριο, μπροστά στο μέτωπο της αναζήτησης.
9. Βάλε την κατάσταση-γονέα στο κλειστό σύνολο.
10. Πήγαινε στο βήμα 3.

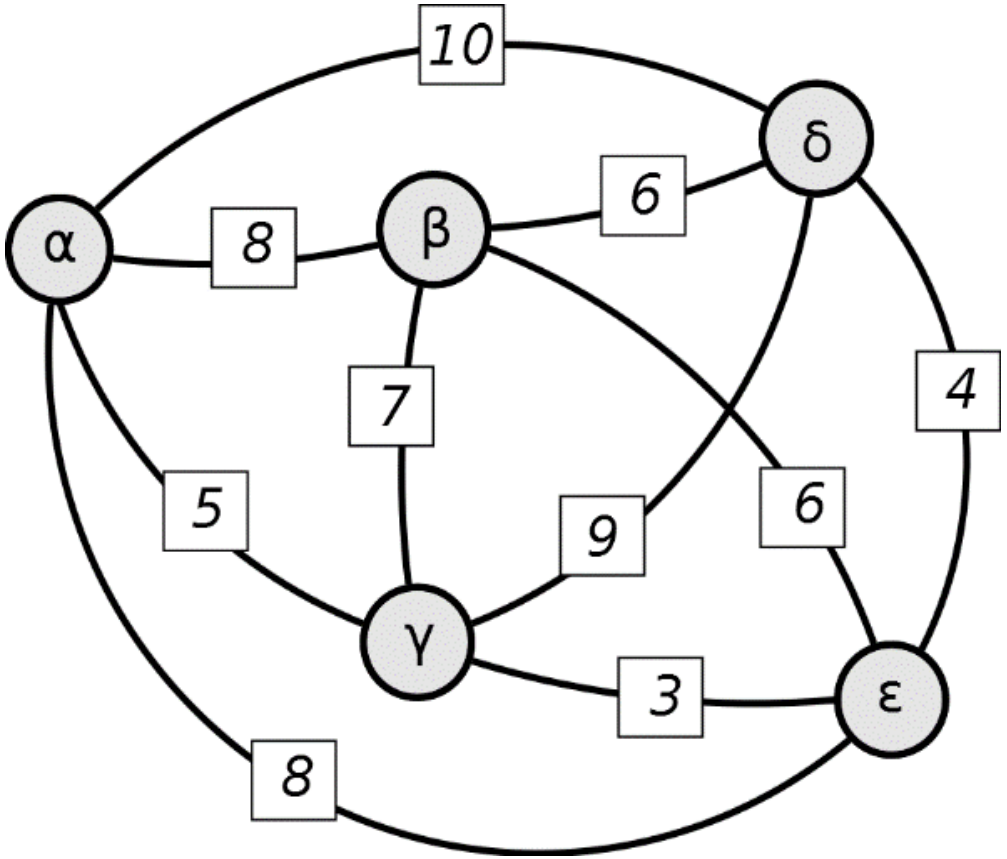
Επέκταση και Οριοθέτηση (B &B)

```
algorithm b&b(InitialState, FinalStates)
begin
 Closed ← ∅;
 Frontier ← <InitialState>;
 BestCost ← o0;
 BestState  ← null;
    while Frontier NOT ∅ do
        CurrentState ← First(Frontier);
        CurrentCost ← Cost(Current_State);
        Frontier ← delete(CurrentState,Frontier);
         if CurrentCost < BestCost then
           if CurrentState ∈ FinalStates then
                BestState ← CurrentState;
                BestCost ← CurrentCost;
                        else
                 Next ← Expand(CurrentState);
        ChildrenStates ←{s| s∈Next ∧ s∉Frontier ∧ s∉Closed};
        Frontier ← ChildrenStates ^ Frontier;
        Closed  ←  Closed{CurrentState};
                                endif;
                        endif;
        endwhile;
                 if BestState = null
                                then return fail
                        else return BestState and BestCost;
        end.
```

Επέκταση και Οριοθέτηση (B &B)

- Εφαρμόζεται σε προβλήματα αναζήτησης Βέλτιστης λύση (ελάχιστο κόστος)
- Συνήθως οι τελεστές μετάβασης δεν έχουν το ίδιο κόστος. Το κόστος διαδρομής είναι μια μονότονη συνάρτηση που αυξάνει καθώς εξελίσσεται η αναζήτηση, επειδή οι τελεστές έχουν πάντα θετικό κόστος

Πλήρης γράφος για το TSP με 5 πόλεις

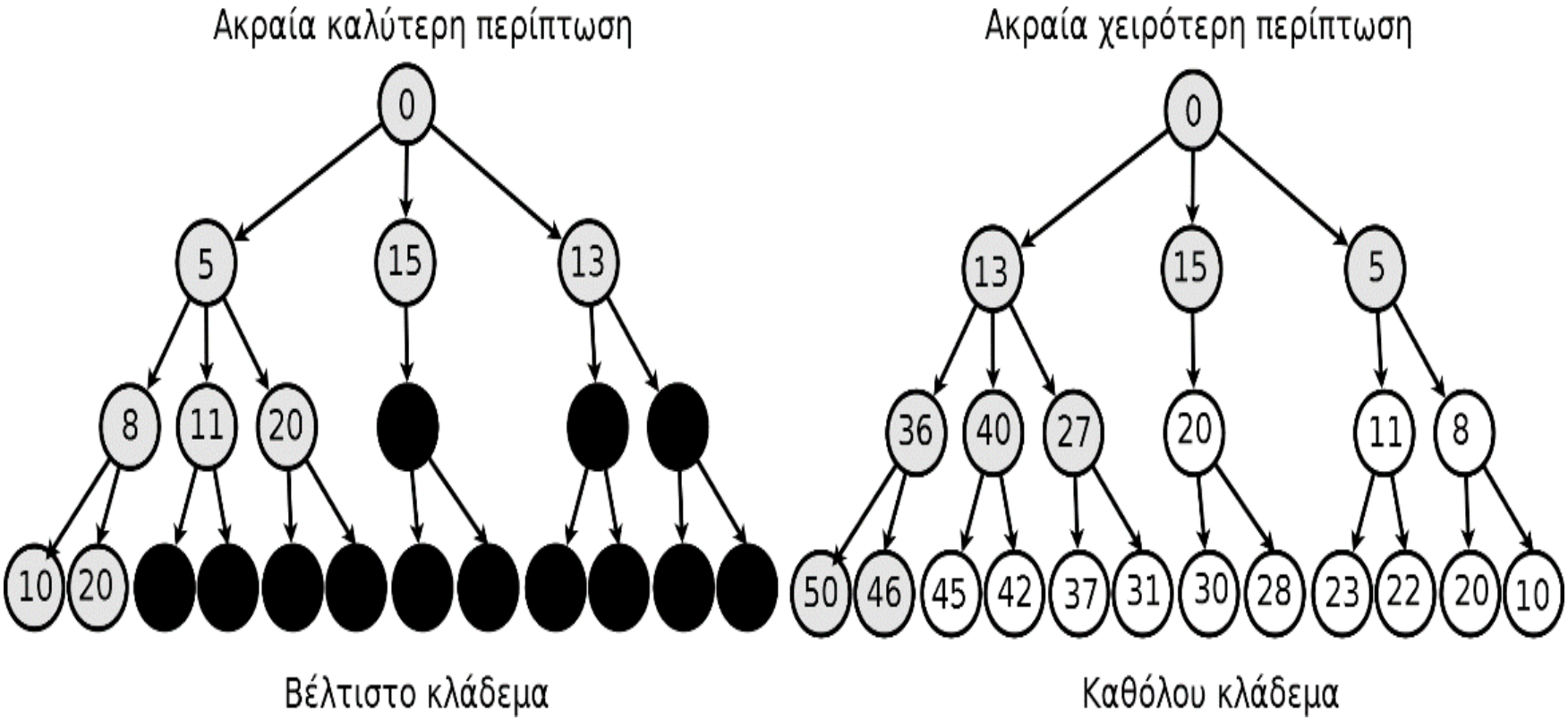


- Μετατρέποντας το γράφο σε Δέντρο έχω 4! Πιθανές λύσεις, διαφορετικού κόστους
- Εξαντλητική Αναζήτηση (N-1)! Μονοπάτια
- Για N-20, έχω $19!=1.216*10^{17}$ λύσεις
- Πρόβλημα με λύση **ΜΗ-πολυωνυμικού χρόνου (NP-complete**)

Βήματα αλγορίθμου B&B στο πρόβλημα TSP.

Μέτωπο της αναζήτησης	Κόστος Λύσης	Κατάσταση	Παιδιά
<α>	+ο0	α	$αβ^{8}$,$αγ^{5}$,$αδ^{10}$,$αε^{8}$
<$αβ^{8}$,$αγ^{5}$,$αδ^{10}$,$αε^{8}$>	+ο0	αβ	$αβγ^{15}$,$αβδ^{14}$, $αβε^{14}$
<$αβγ^{15}$,$αβδ^{14}$,$αβε^{14}$,$αγ^{5}$,...>	+ο0	αβγ	$αβγδ^{24}$, $αβγε^{18}$
<$αβγδ^{24}$,$αβγε^{18}$, $αβδ^{14}$,$αβε^{14}$...>	+ο0	αβγδ	$αβγδε^{28}$
<$αβγδε^{28}$,$αβγε^{18}$, $αβδ^{14}$,...>	+ο0	αβγδε	$αβγδεα^{36}$
< $αβγδεα^{36}$, $αβγε^{18}$, $αβδ^{14}$,..>	36	αβγδεα	Τελική Κατάσταση
<$αβγε^{18}$, $αβδ^{14}$,... >	36	αβγε	$αβγεδ^{22}$
<$αβγεδ^{22}$,$αβδ^{14}$,...>	36	αβγεδ	$αβγεδα^{32}$
< $αβγεδα^{32}$,$αβδ^{14}$,$αβε^{14}$...>	32	αβγεδα	Τελική Κατάσταση
...	...	...	...
<$αβδεγα^{26}$, ...>	26	αβδεγα	Τελική Κατάσταση
...	...	...	...
<$αβεγδ^{26}$, ...>	26	αβεγδ	Κλάδεμα
....	...	...	...
<$αεβγδ^{30}$, ...>	26	αεβγδ	Κλάδεμα
. . .	. . .	. . .	. . .
<>	Ελάχιστη Τιμή	ΤΕΛΟΣ	

Ακραίες περιπτώσεις κλαδέματος από τον αλγόριθμο B&B.



- Η πρώτη καλύτερη λύση με κόστος 10 (Αριστερό Δέντρο)

Επέκταση και Οριοθέτηση (B &B)

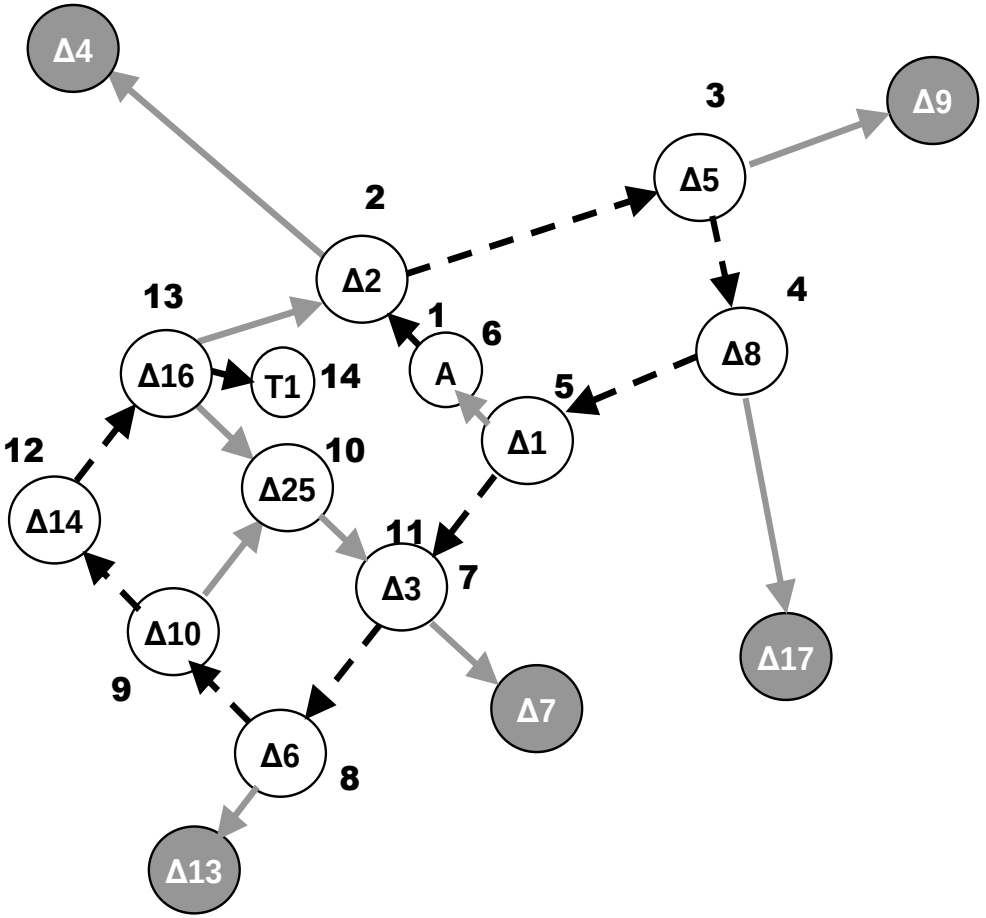
- Το κέρδος από το κλάδεμα καταστάσεων εξαρτάται από το πόσο γρήγορα θα βρεθεί μια καλή λύση.
- Στην χειρότερη περίπτωση ο B&B λειτουργεί όπως ο DFS
- Υπάρχουν διάφορες παραλλαγές. Η πρώτη DFS B&B.
- Ο B&B συνδυάζεται με Δυναμικό Προγραμματισμό, όπου το κλάδεμα δεν γίνεται μόνο σε σύγκριση με το τρέχον όριο (βέλτιστη λύση μέχρι εκείνη τη στιγμή) αλλά και συγκρίνεται με κάθε περιττή κατάσταση.

Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (DD)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων

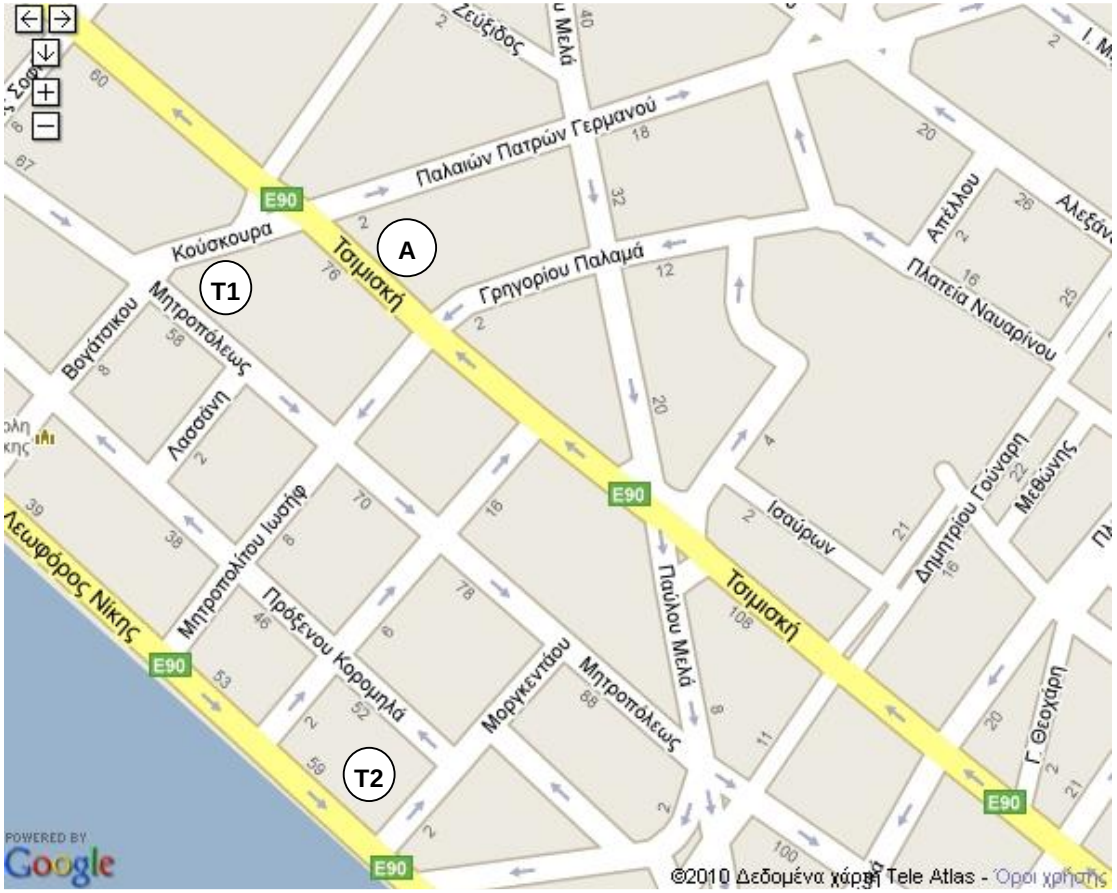
Παράδειγμα

Αεροπορική Εταιρία Αναθέτει Πτήσεις στους Πιλότους της



Ο χώρος αναζήτησης και η λύση του προβλήματος μετά την εφαρμογή του αλγόριθμου DFS και με επιλογή τον δεξιότερο δρόμο που ξεκινά από μία διασταύρωση.

Εύρεση Διαδρομής



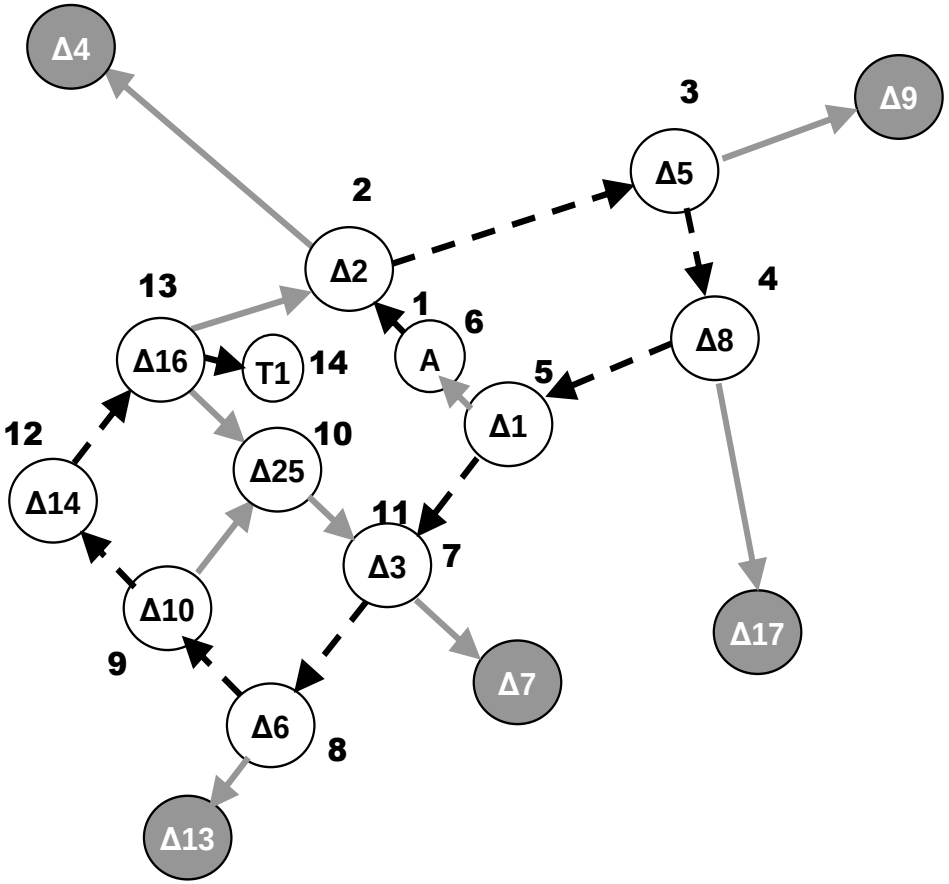
Αυτοκίνητο στο Α
Να μεταβεί Τ1 ή Τ2

Εύρεση Διαδρομής



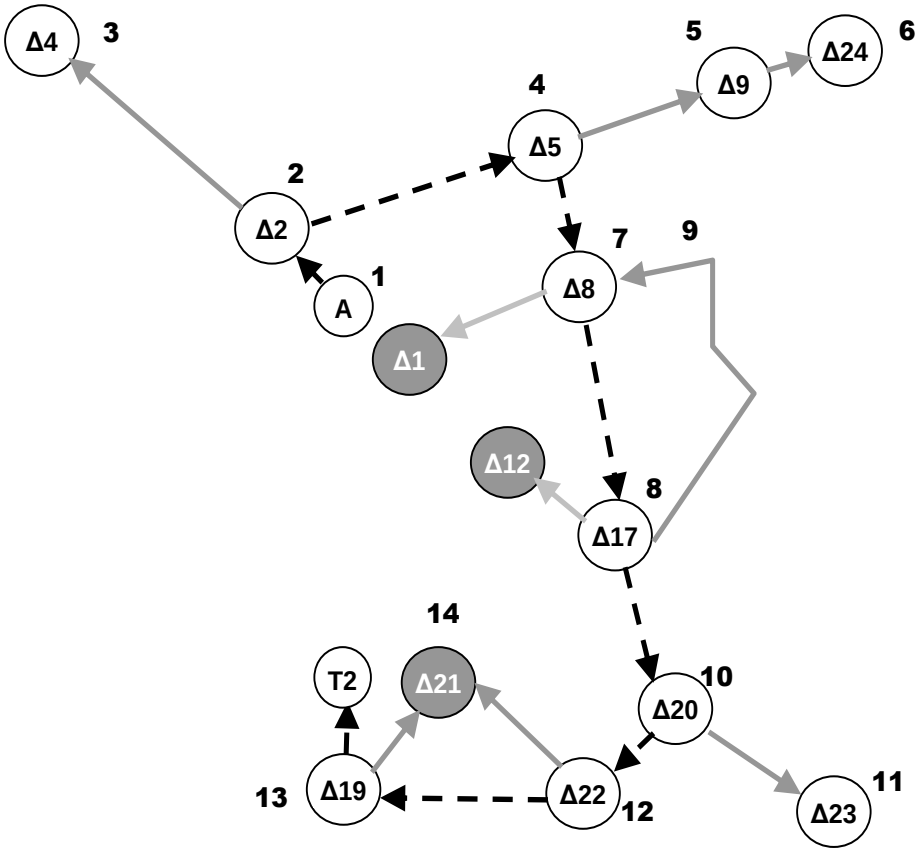
Ο χάρτης ενός μέρους του κέντρου της Θεσσαλονίκης με ταυτόχρονη απεικόνιση του χώρου καταστάσεων.

Εύρεση Διαδρομής

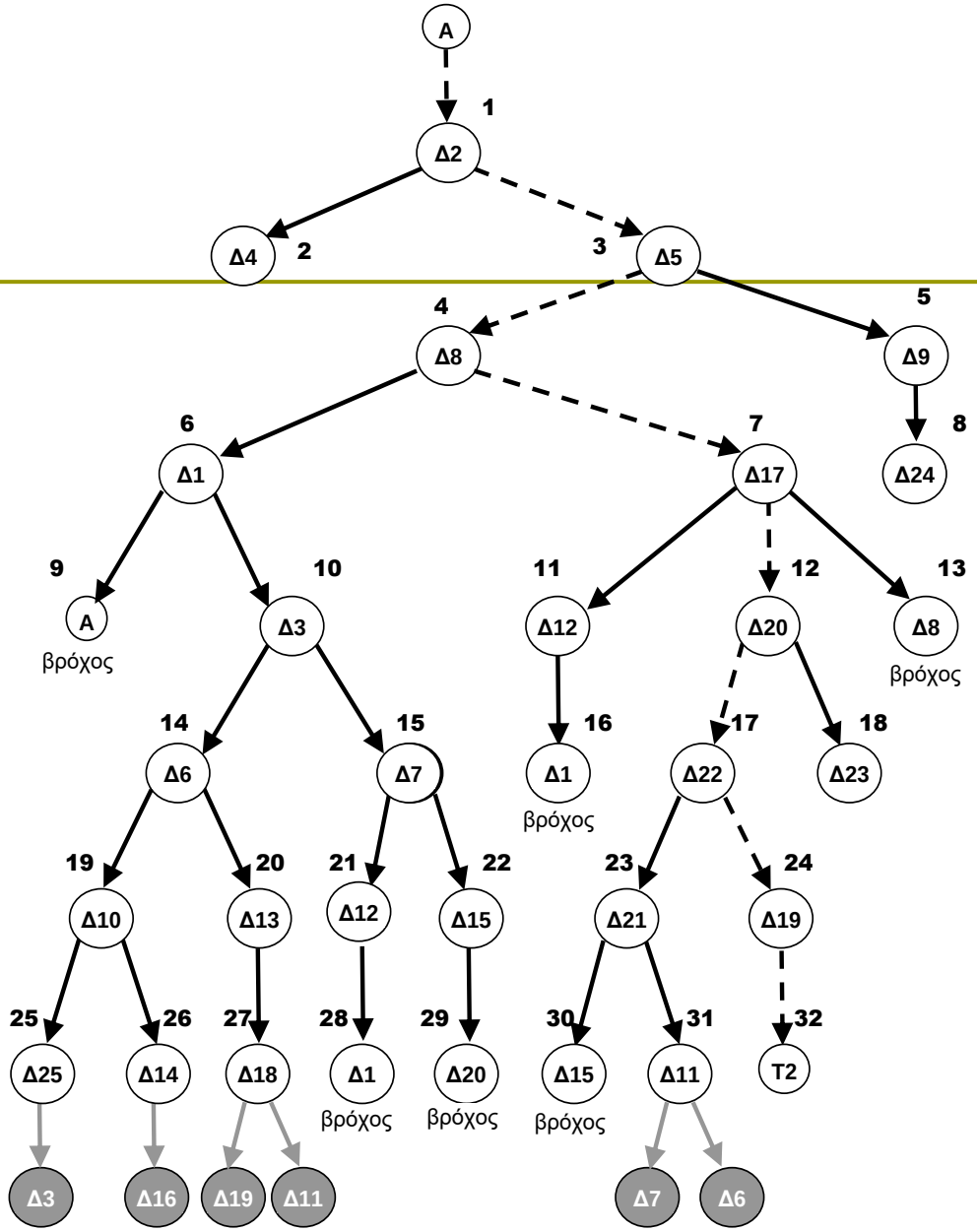


Ο χώρος αναζήτησης και η λύση του προβλήματος μετά την εφαρμογή του αλγόριθμου DFS και με επιλογή τον δεξιότερο δρόμο που ξεκινά από μία διασταύρωση

Εύρεση Διαδρομής

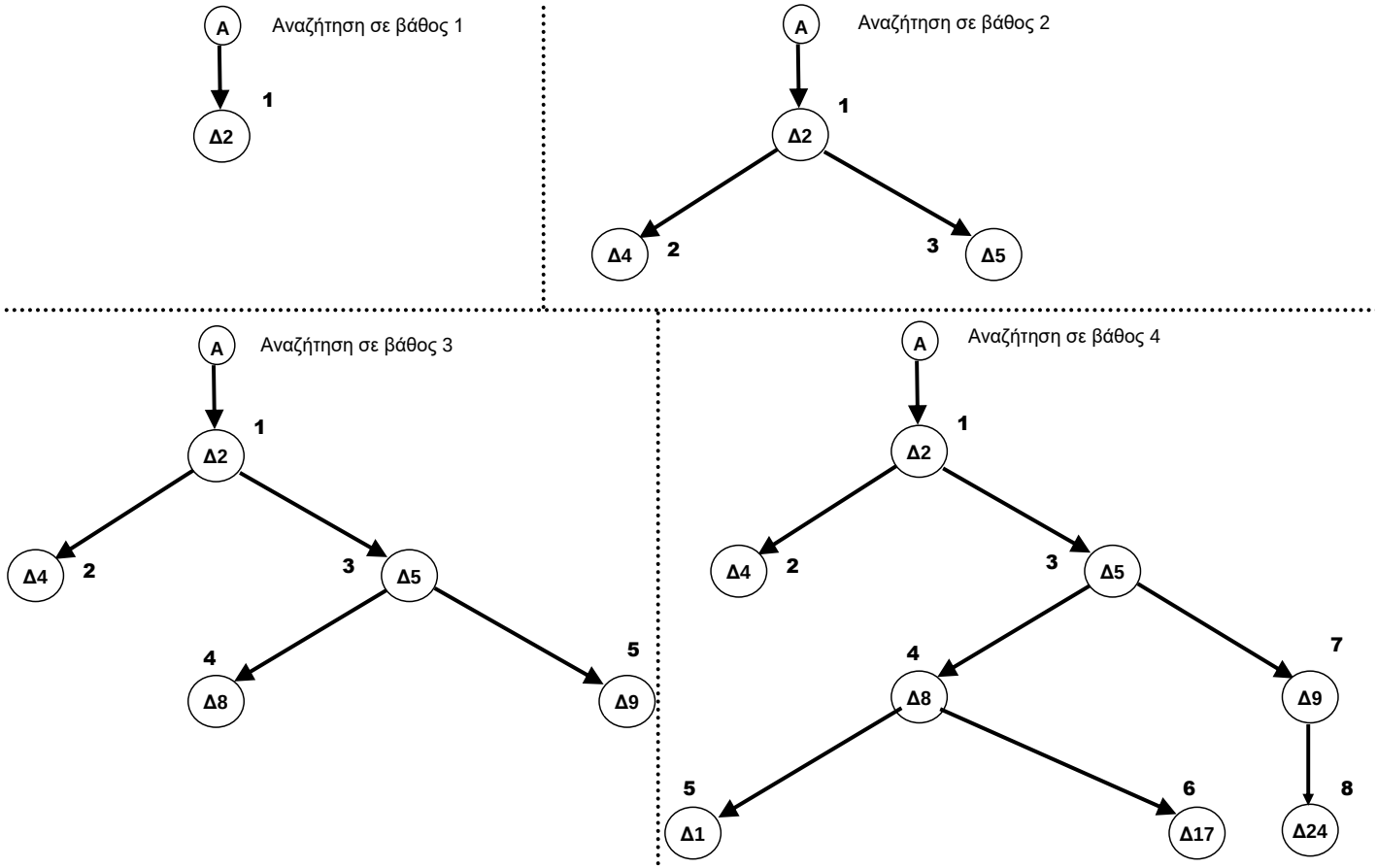


Ο χώρος αναζήτησης και η λύση του προβλήματος με εφαρμογή του DFS και επιλογή του αριστερότερου δρόμου σε κάθε διασταύρωση.



Ο χώρος αναζήτησης και η λύση του προβλήματος με εφαρμογή του BFS και επιλογή του δεξιότερου δρόμου σε κάθε διασταύρωση.

Ο χώρος αναζήτησης κατά την εφαρμογή του αλγορίθμου ID μέχρι βάθος 4.



Ο χώρος αναζήτησης κατά την εφαρμογή του αλγορίθμου ID μέχρι βάθος

Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι Αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
  - Αναζήτηση πρώτα σε βάθος (DFS)
  - Αναζήτηση πρώτα σε πλάτος (BFS)
  - Αλγόριθμος Επαναληπτικής Εμβάθυνσης (ID)
  - Αλγόριθμος Διπλής Κατεύθυνσης (DD)
  - Αλγόριθμος Επέκτασης και Οριοθέτησης (B&B)
- Αλγόριθμοι ευρετικής αναζήτησης
- Παιχνίδια δύο αντιπάλων